UNIVERSIDAD COMPLUTENSE DE MADRID

ntic master
revolucionamos la comunicación

Master Data Science, Big Data & Business Analytics

Fundamentos de Git y GitHub





Rubén López Rodríguez
Javier Domínguez Gómez

Contenido

1. Introducción 4
   1.1. ¿Qué es un sistema de control de versiones? 4
   1.2. Tipos de sistemas de control de versiones 5
   1.3. ¿Qué es Git? 7
   1.4. Fundamentos de Git 8
   1.5. ¿Qué es GitHub? 9
   1.6. ¿Por qué deberías aprender Git y GitHub? 10
2. Trabajando con repositorios locales 12
   2.1. Creación de una cuenta en GitHub 12
   2.2. Instalación de Git 13
   2.3. Configuración inicial de Git 14
   2.4. Crear un repositorio local 14
   2.5. Los tres estados de Git 15
   2.6. Comprobar el estado del repositorio 17
   2.7. Añadir archivos al área de preparación 17
   2.8. Confirmar cambios en el repositorio 19
   2.9. Ver el historial de cambios en el repositorio local 20
   2.10. Recuperar versiones anteriores del repositorio local 20
   2.11. Borrar ficheros del repositorio local 21
3. Trabajando con repositorios remotos 22
   3.1. Crear un repositorio remoto 22
   3.2. Enlazar un repositorio remoto a tu repositorio local 24
   3.3. Enviar cambios al repositorio remoto 25
   3.4. Recuperar cambios del repositorio remoto 27
   3.5. Clonar un repositorio remoto 29
   3.6. Borrar un repositorio remoto 32

4. Trabajando con Entornos de desarrollo 33
4.1. IDEs más utilizados en Python 33
4.2. Instalación de Python y Visual Studio Code 36
4.3. Crear un proyecto de Python 38
4.4. Crear un repositorio local 39
4.5. Crear un repositorio remoto 41
5. Trabajando con Ramas 44
5.1. ¿Qué son las ramas (branches) en Git? 44
5.2. Creación de ramas 46
5.3. Fusionar ramas (merge) 48
5.4. Resolviendo conflictos 52
5.5. Ramas y repositorios remotos 56
6. Más sobre Git y GitHub 60
6.1. Buenas prácticas de uso de Git 60
6.2. Gitignore: Ignorar archivos y directorios 60
6.3. Etiquetas (tags) 61
6.4. Estrategias de ramas en Git 63
6.5. Colaboración en equipo (fork y pull requests) 66
6.6. GitHub Pages 71
6.7. Learn Git Branching 73

1. Introducción

El objetivo de este módulo es proporcionar al alumno un conocimiento básico para comenzar a utilizar **Git** y **GitHub**. Veremos qué son los sistemas de control de versiones, trabajaremos con repositorios locales y, a continuación, con repositorios remotos. De esta manera aprenderemos a registrar los cambios realizados en nuestros proyectos particulares y realizar contribuciones en otros proyectos.

Pero, antes de hablar de Git o GitHub, tenemos que entender qué es un sistema de control de versiones.

1.1. ¿Qué es un sistema de control de versiones?

Un sistema de control de versiones (VCS, por sus siglas en inglés, Version Control System) es un software que registra los cambios realizados en documentos a lo largo del tiempo. Su principal función es permitirte volver a una versión anterior de tus archivos en cualquier momento. Esto te permite tener un histórico de todos los cambios producidos en sus ficheros, saber quién los hizo y cuándo.

Podemos verlo como un sistema para realizar copias de seguridad. Pero nos quedaríamos muy cortos, ya que un sistema de control de versiones también nos permite, por ejemplo, sincronizar cambios de diferentes personas y trabajar en diferentes versiones a la vez.

Los sistemas de control de versiones son esenciales en el desarrollo de software, pero también son increíblemente útiles para cualquier tipo de trabajo que implique la creación y edición de archivos (principalmente de texto). Estas son algunas de sus ventajas clave:

- **Colaboración**: Permite que múltiples personas trabajen en el mismo proyecto al mismo tiempo sin sobrescribir los cambios de los demás. El sistema se encarga de fusionar el trabajo de todos.

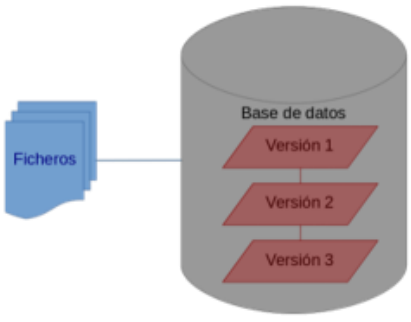
- **Historial detallado**: Mantiene un registro de quién, cuándo y por qué se realizó cada cambio. Esto facilita la auditoría y la comprensión de la evolución del proyecto.
- **Recuperación de errores**: Si un archivo se daña o se elimina por accidente, puedes restaurar una versión anterior fácilmente.
- **Experimentación segura**: Puedes crear "ramas" (branches) de tu proyecto para probar nuevas ideas sin afectar la versión principal. Si la nueva idea funciona, puedes integrarla; si no, simplemente la descartas.

1.2. Tipos de sistemas de control de versiones

Hay tres tipos principales de sistemas de control de versiones, que han evolucionado a lo largo del tiempo para adaptarse a las necesidades de colaboración y a las arquitecturas de software modernas:

1. Sistemas de Control de Versiones Locales (LVCS)

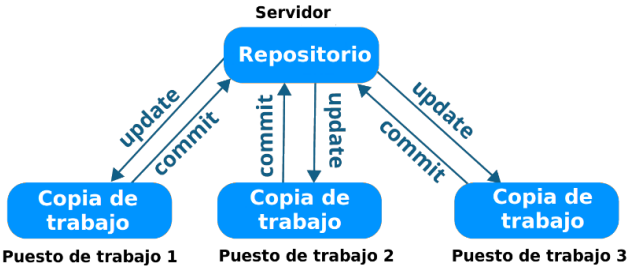
Estos fueron los primeros y más sencillos. Funcionaban directamente en una sola máquina. En lugar de un sistema centralizado, el control de versiones se basaba en una base de datos local donde el desarrollador almacenaba las revisiones de sus archivos.



Ejemplos: RCS (Revision Control System) y SCCS (Source Code Control System).

2. Sistemas de Control de Versiones Centralizados (CVCS)

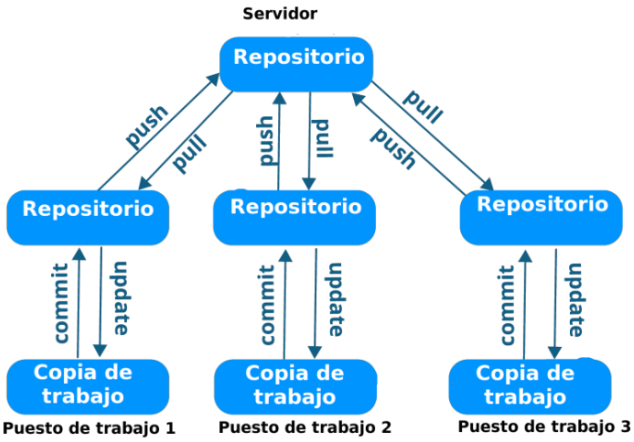
Para resolver los problemas de colaboración y de respaldo de los LVCS, surgieron los sistemas centralizados. En un CVCS, hay un servidor central único que contiene el repositorio principal del proyecto, incluyendo el historial completo de todas las versiones.



Ejemplos: SVN (Subversion) y CVS (Concurrent Versions System).

3. Sistemas de Control de Versiones Distribuidos (DVCS)

Estos sistemas son la evolución más reciente y el estándar actual en la industria. A diferencia de los CVCS, en un DVCS no hay un único servidor central que sea la "fuente de verdad" del proyecto. En ellos cada usuario tiene una copia completa de todo el historial del proyecto en local. Además, existe un servidor central remoto que contiene el repositorio maestro.



Ejemplos: Git (el más popular), Mercurial y Darcs.

1.3. ¿Qué es Git?

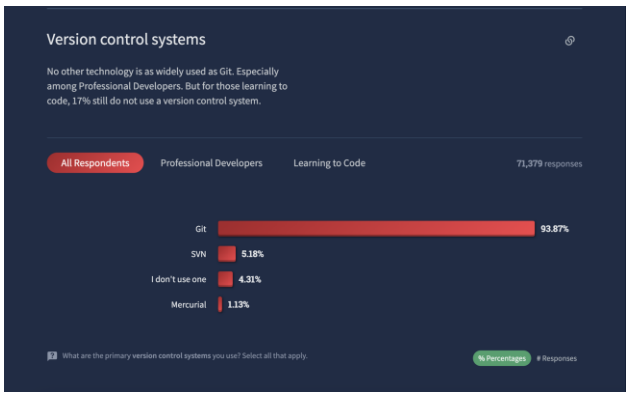
Se trata de un sistema de control de versiones distribuido, gratuito y de código abierto, ampliamente utilizado en el desarrollo de software. Permite a los equipos de programadores colaborar en proyectos, realizar un seguimiento de los cambios en el código fuente y mantener un historial completo de las modificaciones. Una de las principales características de Git es su capacidad para gestionar múltiples ramas de desarrollo de forma eficiente.

Fue desarrollado por Linus Torvalds, el creador del kernel Linux. La historia de Git se remonta a principios de los 2000 cuando la comunidad de desarrolladores de GNU/Linux necesitaba una alternativa eficiente y robusta al sistema de control de versiones existente en ese momento, llamado BitKeeper.

Cuando BitKeeper, que era utilizado para gestionar el desarrollo de GNU/Linux, dejó de ser gratuito para grandes proyectos de código abierto, Linus Torvalds decidió crear su propio sistema de control de versiones para cubrir las necesidades de la comunidad de GNU/Linux y otros proyectos de código abierto. En abril de 2005, anunció oficialmente el proyecto Git.

Desde entonces, Git ha ido ganando popularidad no solo en la comunidad de desarrolladores de GNU/Linux, sino en toda la industria del desarrollo de software, debido a su flexibilidad, rendimiento y confiabilidad, hasta convertirse en el estándar de facto.

Hoy en día se trata de la herramienta de control de versiones de software más utilizada del mundo. En 2022 la web de consultas sobre desarrollo stack overflow realizó una encuesta sobre los sistemas de control de versiones más utilizado. Git arrasó con más de un 90%.



https://survey.stackoverflow.co/2022/#section-version-control-version-control-systems

Desde entonces no han vuelto a realizar dicha consulta ya que el dominio de Git es absoluto.

1.4. Fundamentos de Git

Como se ha visto con anterioridad Git es un sistema de control de versiones **distribuido** y su base son los **repositorios**. Un repositorio es un directorio especial donde Git almacena de forma segura todo el historial de cambios de tu proyecto, incluyendo los archivos, las versiones y los metadatos de cada modificación (quién la hizo, cuándo y por qué).

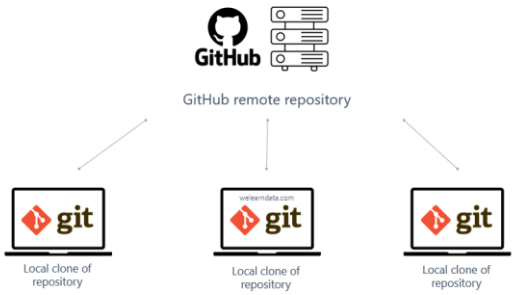
En este punto es fundamental entender que, cuando utilizamos Git, los repositorios pueden ser locales o remotos:

- **Repositorio Local**: Es el repositorio que se encuentra en tu propia máquina, en tu PC. Contiene una copia completa del historial de cambios y archivos de tu proyecto local. Por ejemplo:

  C:/users/tu_usuario/carpeta_del_proyecto

- **Repositorio Remoto**: Este es un repositorio en un servidor remoto (GitHub, GitLab o BitBucket) en internet al que varios desarrolladores pueden acceder. Es útil para la colaboración en equipo y para respaldar el código. Por ejemplo:

  https://github.com/tu_usuario/mi-proyecto.git



Además, cada repositorio tendrá al menos una **rama principal**. Esta rama, actualmente, suele llamarse “main”. De esta rama principal pueden salir otras ramas o bifurcaciones (branches) con diferentes finalidades.

De esta manera los diferentes cambios que realicemos en nuestros archivos pasarán a formar parte del historial/repositorio local y, después, empujaremos dichos cambios al repositorio remoto. Además, dichos cambios serán almacenados en una rama concreta, pudiendo lugar pasar a integrarse con otras ramas, en un proceso llamado **merge**.

1.5. ¿Qué es GitHub?

GitHub es una plataforma de desarrollo colaborativo y un servicio de alojamiento de repositorios de código que utiliza el sistema de control de versiones Git. En términos sencillos, es un servicio basado en la nube que permite a los desarrolladores y equipos almacenar, gestionar y colaborar en proyectos de software de forma eficiente.

A diferencia de Git, que es la herramienta de control de versiones en sí misma, GitHub es la plataforma que proporciona una interfaz gráfica y una serie de funcionalidades adicionales sobre Git, facilitando su uso y la colaboración a gran escala.

Es decir, mientras que **Git** es la herramienta fundamental para el control de versiones, **GitHub** es la plataforma que la aloja en la nube y añade una capa de colaboración y gestión de proyectos de desarrollo de software.

Características y funciones clave de GitHub:

- **Alojamiento de repositorios**: Permite almacenar proyectos de código (conocidos como repositorios) en la nube. Los proyectos pueden ser públicos (para que cualquiera los vea y contribuya) o privados.

- **Colaboración**: Facilita el trabajo en equipo a través de herramientas como las "pull requests", que permiten a los desarrolladores proponer cambios a un proyecto y discutirlos antes de fusionarlos con el código principal.

- **Gestión de proyectos**: Ofrece herramientas para la gestión de tareas, seguimiento de errores ("issues"), y wikis, ayudando a los equipos a planificar y organizar su trabajo.

- **Comunidad y red social**: Funciona como una red social para desarrolladores, permitiendo seguir a otros programadores, explorar proyectos de código abierto y construir un portafolio de trabajo.

- **Automatización**: Proporciona herramientas de integración y despliegue continuo (CI/CD) a través de GitHub Actions, que automatizan tareas como la compilación, prueba y despliegue de código.

- **GitHub Pages**: Permite crear sitios web estáticos directamente desde un repositorio, ideal para blogs o páginas de documentación de proyectos.

Aunque existen otras plataformas de desarrollo colaborativo tales como GitLab o Bitbucket, GitHub es, de largo, la más popular y utilizada, como demuestra la pregunta que stack overflow lanzó en su encuesta de 2022 sobre las plataformas de control de versiones más populares. En dicha encuesta GitHub estuvo cerca del 90%.



https://survey.stackoverflow.co/2022/#version-control-version-control-system

1.6. ¿Por qué deberías aprender Git y GitHub?

Para alguien que quiera empezar una carrera en el mundo del desarrollo, en cualquiera de sus variantes, aprender Git y GitHub no es opcional: es prácticamente parte del idioma que se habla en cualquier equipo técnico.

Estas son algunas de las razones:

1. Es un **estándar** de la industria
    - Git es el sistema de control de versiones más usado del mundo, y GitHub es una de las plataformas de colaboración más populares.
    - La mayoría de las empresas asumen que cualquier desarrollador sabe usarlo, igual que esperarían que sepas escribir un for en tu lenguaje de programación.

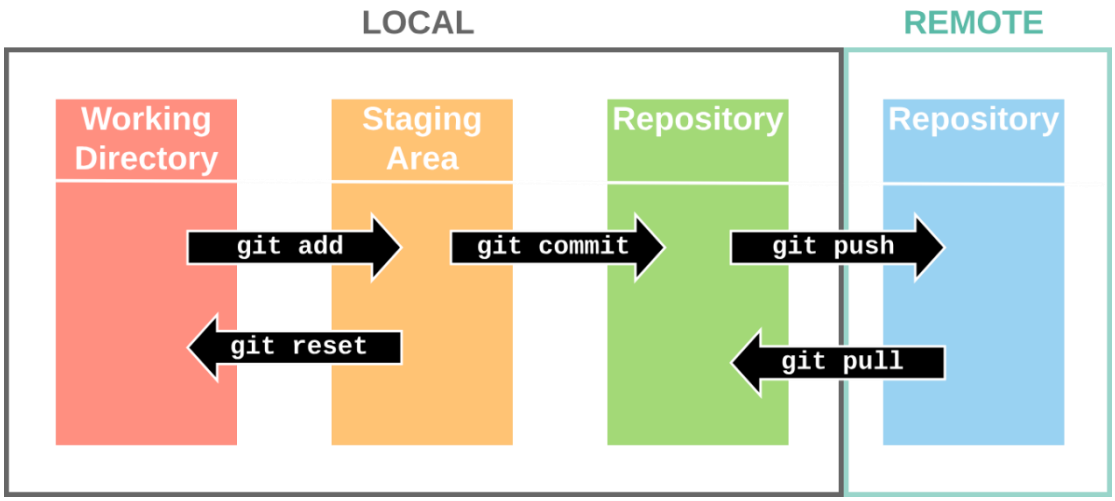
2. **Portafolio** profesional
    - GitHub funciona como una vitrina pública de tus proyectos y habilidades.
    - Tener un perfil activo y organizado puede impresionar a reclutadores y empleadores.

3. Prácticas de desarrollo **modernas**
    - Git es clave para aplicar metodologías como Git Flow, Integración Continua y Despliegue Continuo (CI/CD).
    - Es parte de pipelines automatizados en empresas de todos los tamaños.

4. **Colaboración** global
    - Muchas comunidades open source usan GitHub como base de operaciones.
    - Conocerlo te permite contribuir a proyectos de cualquier parte del mundo, lo que es una excelente forma de aprender y hacer networking.

2. Trabajando con repositorios locales

Este capítulo te guiará para empezar a trabajar con Git directamente en tu computadora, sin necesidad de conectarte a un repositorio remoto. Es una excelente manera de empezar a versionar tu trabajo, entender los conceptos fundamentales de Git y tener un control de tus archivos localmente. Aquí, cubriremos el proceso de cómo iniciar un nuevo proyecto y nos enfocaremos en las primeras tres fases de un flujo de trabajo típico de Git.

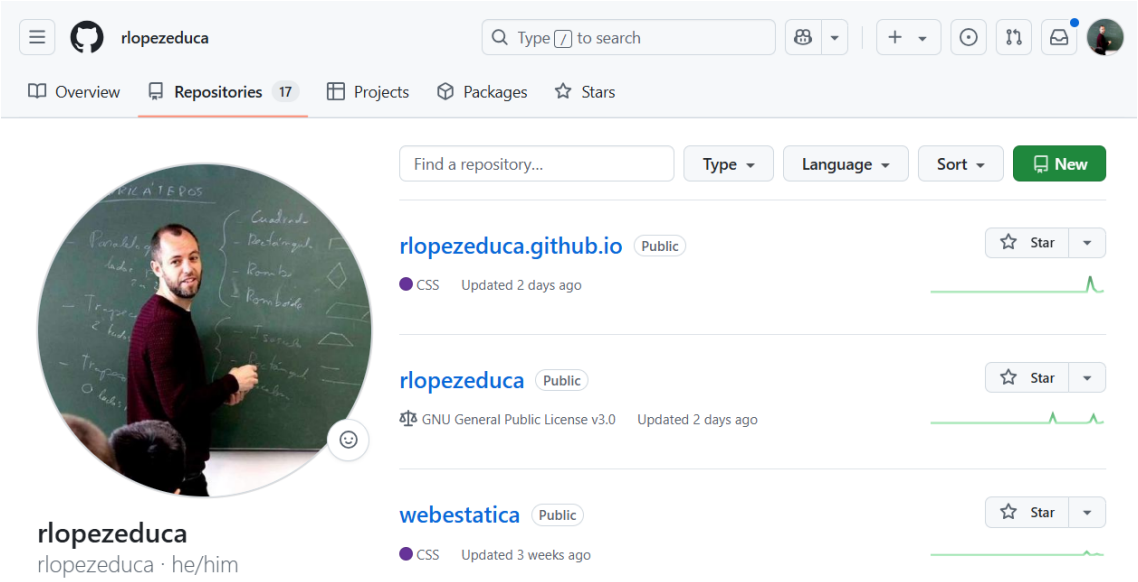


2.1. Creación de una cuenta en GitHub

Antes de empezar a trabajar con repositorios locales es recomendable tener una cuenta en GitHub, GitLab, Bitbucket u otra plataforma de desarrollo colaborativo para poder interactuar con Git en proyectos públicos o privados en la red. Estos proyectos también se llaman repositorios o “repos”.

En este módulo vamos a trabajar con GitHub, que es el más popular y utilizado por la comunidad. GitHub proporciona una interfaz web que hace que Git sea más fácil de usar, y permite a los desarrolladores colaborar en proyectos desde cualquier lugar del mundo.

Para realizar los ejercicios prácticos de este módulo, es necesario disponer de una cuenta en GitHub. Para ello entra en la página oficial de GitHub y crea tu propia cuenta. Adicionalmente podemos personalizar nuestro perfil añadiendo una foto de perfil o nuestro nombre, alias, etc.



En la creación de la cuenta de GitHub vas a tener que facilitar un **correo electrónico**, este es único por usuario, y un **nombre de usuario**. Vas a necesitar ambos datos más adelante para configurar tu usuario en la herramienta Git.

2.2. Instalación de Git

Para comenzar a usar Git, primero debes instalarlo en tu sistema. Los pasos varían según el sistema operativo que estés utilizando. Descarga el instalador de Git desde el sitio web oficial y sigue las instrucciones de instalación.

Después de la instalación, verifica que Git Bash se haya instalado correctamente y ejecuta el siguiente comando en una terminal:

```
git --version
```

Puedes consultar el "**ANEXO I. Instalación de Git**" para más detalles.

2.3. Configuración inicial de Git

Antes de empezar a utilizar Git, es importante configurar tu identidad. Esto se hace mediante los siguientes comandos, reemplazando "tu_nombre" y "tu_correo@email.com" con tu nombre y la dirección de correo electrónico que has usado para registrar tu cuenta en GitHub:

```
git config --global user.name "tu_nombre"
git config --global user.email "tu_correo@email.com"
```

Estos valores se utilizarán para identificar tus contribuciones en los registros de Git. Puedes verificar la configuración ejecutando:

```
git config --list
```

o

```
git config --global --list
```

2.4. Crear un repositorio local

Para comenzar a utilizar Git lo primero que debes hacer es crear un repositorio. Esta acción creará una carpeta, el “working directory”, donde se almacenarán todos los ficheros de tu proyecto. Para ello utilizamos el comando siguiente:

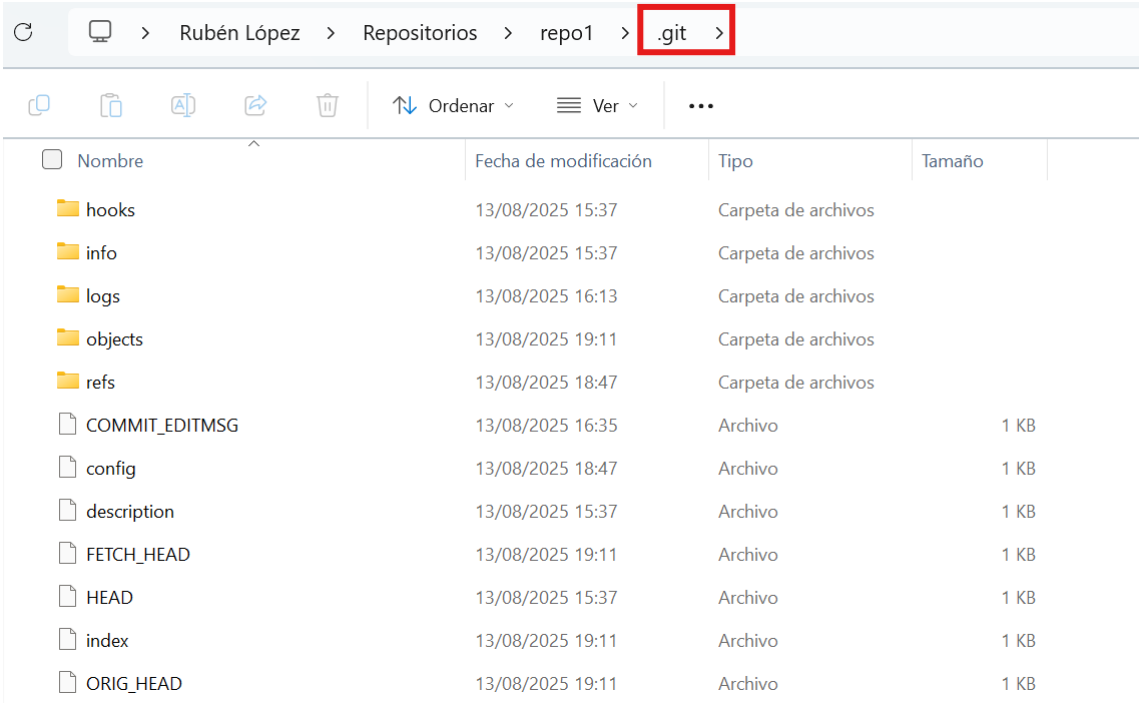
```
$ git init repo1
Inicializado repositorio Git vacío en C:/Users/tu_usuario/repo1/.git/
```

También podemos ejecutar el comando git init sobre una carpeta ya existente. En este caso no creará la carpeta, debido a que ya existía, pero sí la convertirá en un repositorio:

```
$ mkdir repo1
$ git init repo1
Inicializado repositorio Git vacío en C:/Users/tu_usuario/repo1/.git/
```

En ambos casos se creará una carpeta oculta (aquella cuyo nombre comienzan con un punto) llamada “**.git**”. Es muy importante que esta carpeta no la borremos, ya que contendrá toda la información sobre nuestro proyecto en Git.

Podemos inspeccionar qué estructura de directorios y archivos tiene esta carpeta "**.git**" con abriendo la misma desde el Explorador de archivos:



En este momento nuestro repositorio está vacío, todavía no tiene archivos creados por nosotros. Pero antes de eso vamos a hablar de los estados de Git, para entender qué ocurre cuando creamos archivos dentro de nuestro repositorio.

2.5. Los tres estados de Git

Los archivos de tu proyecto, los que están ubicados dentro de tu "Working Directory", pueden existir en tres estados principales. Entender estos estados es fundamental para usar Git de manera efectiva, ya que te ayuda a saber qué cambios se están registrando y cuáles no.

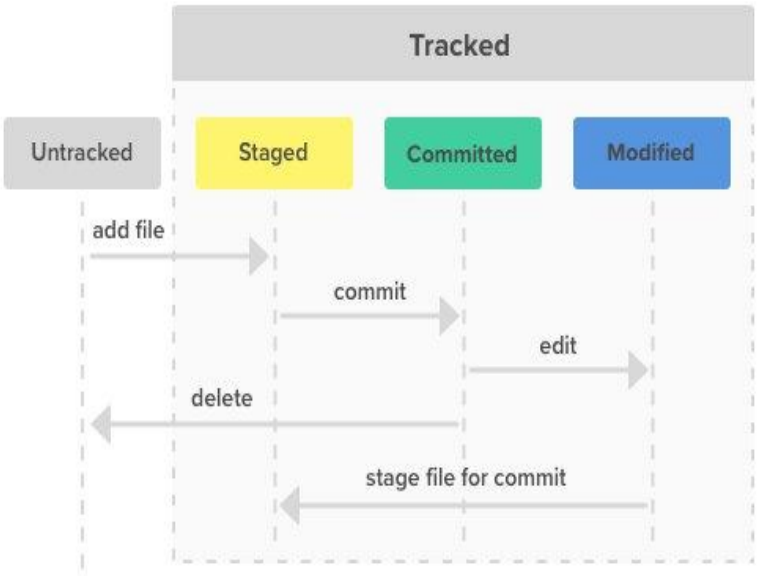
Los tres estados son:

- **Preparado** (Staged): Significa que los cambios en un archivo han sido marcados para ser incluidos en la próxima confirmación (commit). Estos cambios se encuentran en el área de preparación (staging area), listos para ser confirmados.

- **Confirmado** (Committed): Indica que los cambios han sido guardados de forma segura en el repositorio local de Git. Los cambios han sido confirmados con un mensaje descriptivo y forman parte del historial del proyecto.

- **Modificado** (Modified): Indica que un archivo ha sido cambiado desde la última vez que fue confirmado en Git. Los cambios no se han añadido al área de preparación (staging area) aún.

Además, los archivos podrían encontrarse **fuera de seguimiento** (Untracked) cuando se acaban de crear y aún no se han preparado para la próxima confirmación.



El flujo de trabajo básico en un repositorio local Git es, en esencia, el siguiente:

1. Creas o modificas una serie de archivos en tu directorio de trabajo.
2. Etiquetas los archivos, añadiéndolos a tu área de preparación.
3. Confirmas los cambios haciendo un commit. Esto toma los archivos tal y como están en el área de preparación y almacena una “fotografía” de los mimos en su estado actual en el directorio .git.

2.6. Comprobar el estado del repositorio

Para comprobar en qué estado se encuentran los ficheros de un repositorio podemos utilizar el siguiente comando:

```
git status
```

Por ejemplo, si creamos un nuevo fichero en nuestro repositorio llamado mensaje.txt y ejecutamos el comando git status este es el resultado:

```
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        mensaje.txt

nothing added to commit but untracked files present (use "git add" to
track)
```

2.7. Añadir archivos al área de preparación

Ahora que ya tenemos un repo local y entendemos los tres estados de Git podemos comenzar a realizar cambios en nuestros archivos locales. El proceso típico implica los siguientes pasos:

En primer lugar, crearemos o modificaremos archivos en el Working Directory. Por ejemplo, vamos a crear un archivo nuevo llamado "**mensaje.txt**":

```
touch mensaje.txt
```

Una vez creado el archivo, lo abrimos y lo editamos con nuestro editor favorito (PyCharm, VSCode, nano, block de notas, etc), por ejemplo haciendo doble click con el ratón sobre el archivo recién creado en la carpeta, de este modo lo abriremos con un editor de texto standard.

En este momento podemos ejecutar el comando "**git status**" para ver qué ha sucedido y cuál es el estado actual de nuestro repo:

```
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        mensaje.txt

nothing added to commit but untracked files present (use "git add" to
track)
```

Ahora sí, podemos agregar ficheros deseados al staging area con el comando "**git add**"

```
git add mensaje.txt
```

Si hemos modificado varios ficheros podemos utilizar el siguiente comando para agregar todos los ficheros con cambios al staging area. De esta manera no tenemos que agregar fichero a fichero.

```
git add .
```

Ahora volvemos a ejecutar el comando "**git status**", para ver la diferencia respecto a la vez anterior que lo hemos ejecutado. Ahora veremos que ya hemos añadido un archivo nuevo a nuestro staging area y está listo para comitear.

```
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   mensaje.txt
```

Si queremos eliminar un archivo del staging area, que aún no ha sido comiteado, podemos utilizar el siguiente comando:

```
git restore --staged <nombre_archivo>
```

2.8. Confirmar cambios en el repositorio

Ya hemos creado o modificados ficheros del “Working directory” y los hemos agregado al staging area. Es momento de realizar nuestro primer **commit**. Un commit es una instantánea de los cambios que has guardado, hasta el momento, en tu repositorio local. Es como si estuvieras haciendo una "fotografía" del estado actual de tu proyecto.

Para guardar los cambios utilizaremos el siguiente comando, con un mensaje descriptivo, “**git commit -m <texto-descriptivo-del-cambio>**”.

```
$ git commit -m "Primera versión"
[main (root-commit) 0bf8939] Primera versión
1 file changed, 1 insertion(+)
create mode 100644 mensaje.txt
```

Es importante entender que, cada vez que realizamos un commit, guardamos, en el repositorio local todos los cambios de los ficheros del staging area. Es decir, un commit no se realiza sobre un fichero en concreto, si no sobre todos los archivos que estaban en espera de confirmación (staging area).

Ahora podríamos volver a ejecutar el comando “**git status**”, para ver la diferencia respecto a la vez anterior que lo hemos ejecutado. Nos indica que no hay ningún cambio que comitear, ya que acabamos de hacerlo.

```
$ git status
On branch main
nothing to commit, working tree clean
```

Podemos ver que todos los cambios del repositorio han sido confirmados, ya que nos indica que no queda nada por comitear.

También es posible realizar los dos pasos anteriores, añadir todos los ficheros modificados al staging area y realizar un commit de los cambios, con un solo comando. Aunque esta opción no funcionará para archivos de nueva creación.

```
$ git commit -am "Primera versión"
```

Si, una vez realizado el commit, nos arrepentimos y queremos deshacerlo podremos utilizar el siguiente comando:

```
$ git reset --soft HEAD~1
```

Al ejecutar este comando se elimina el commit del historial de git, pero mantiene los archivos con sus modificaciones en la staging area.

Se podría utilizar la versión “hard” de este comando. En este caso eliminaría también todos los cambios de los archivos que se incluyeron en el commit.

2.9. Ver el historial de cambios en el repositorio local

Ya hemos creado nuestro repo local y hemos realizado cambios en el mismo. Ahora vamos a consultar el historial de versiones de nuestro repo:

Para ver el historial de cambios ejecutamos el siguiente comando.

```
$ git log
```

También podemos utilizar el comando siguiente, que muestra los identificadores de cada commit de manera resumida.

```
$ git log --oneline
```

2.10. Recuperar versiones anteriores del repositorio local

Una vez que tenemos varias versiones de nuestro archivo en el repositorio local estamos preparados para restaurar versiones anteriores. Supongamos que hemos modificado un archivo del Working directory.

Si queremos ver las **diferencias del archivo actual** con la última versión del repositorio utilizamos el siguiente comando:

```
git diff <nombre_archivo>
```

Supongamos que nos hemos equivocado y queremos anular las modificaciones realizadas y recuperar la última versión del repositorio. Podemos utilizar el siguiente comando para restaurar la **última versión de un archivo** y anular los cambios actuales.

```
git restore <nombre_archivo>
```

También podemos **recuperar una versión concreta** de un archivo, por ejemplo, la primera versión. Para ello, primero mostramos el historial de versiones y copiamos el id de la versión que nos interesa. Después, ejecutamos el siguiente comando:

```
git restore --source=<id_version> <nombre_archivo>
```

Por último, también podemos "navegar" hasta un commit anterior al actual. Esto hace que todos los archivos de tu proyecto se muestren como estaban en dicho momento, sería como "viajar en el tiempo". Pero esto coloca tu repositorio en un estado de "detached HEAD", lo que significa que no estás en ninguna rama. Tendrías que crear una nueva rama a partir de este commit para poder realizar cambios en tu repositorio.

Para navegar a un commit anterior utilizamos el siguiente comando:

```
git checkout <id_version>
```

Para volver a la rama principal habría que utilizar el siguiente comando:

```
git checkout main
```

2.11. Borrar ficheros del repositorio local

Es posible que, después de trabajar durante un tiempo en tu repositorio local, desees borrar algún fichero que ya esté subido al mismo.

Para ello podrías borrar el fichero de tu carpeta, añadir dicho cambio al staging area y, por último, confirmar los cambios.

Otra opción más cómoda sería utilizar el comando git rm. Para ello sigue los pasos siguientes:

1. Borra el fichero de tu carpeta y del área de seguimiento:

```
git rm <nombre_archivo>
```

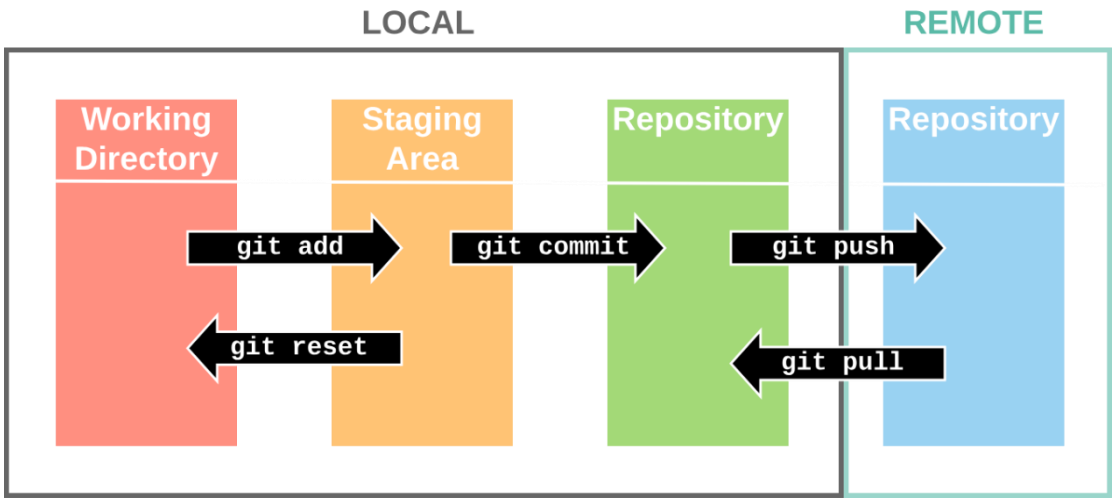
2. Confirma el cambio anterior:

```
git commit -m "mensaje"
```

3. Trabajando con repositorios remotos

Hasta el momento hemos trabajado con repositorios locales, lo que limita la visibilidad de los cambios a nuestro propio equipo. Si bien esto puede ser adecuado para proyectos personales, la colaboración en equipo requiere un método para compartir y sincronizar el trabajo.

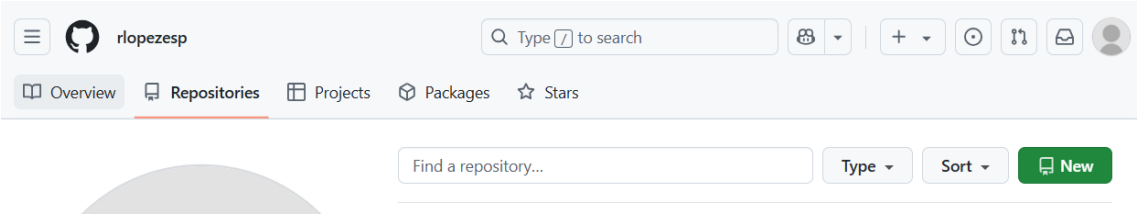
Para abordar esta necesidad, se utilizan los repositorios remotos (por ejemplo, GitHub). Estos repositorios residen en un servidor externo y facilitan la comunicación de los cambios entre múltiples repositorios locales, sirviendo como un punto de sincronización central.



3.1. Crear un repositorio remoto

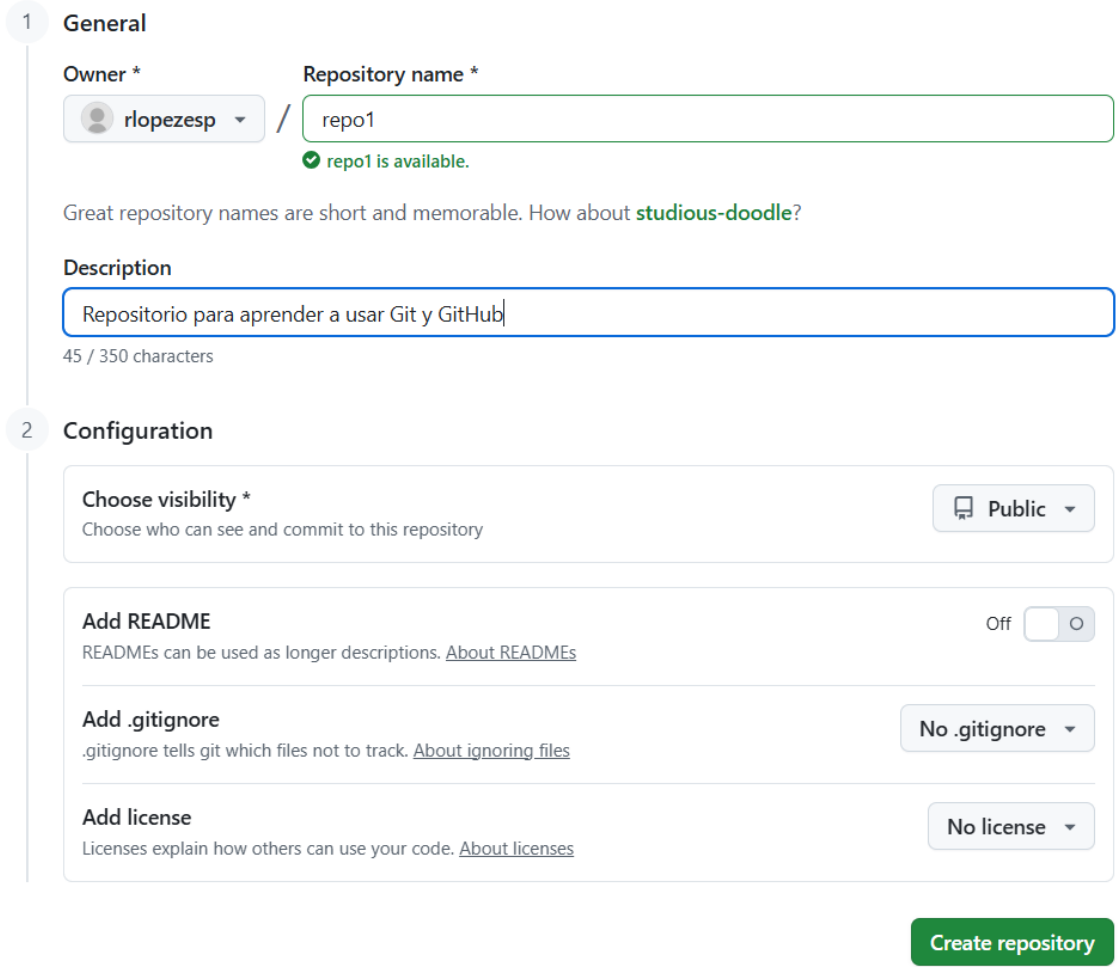
Ahora que ya tenemos un repositorio local llamado "**repo1**", con algún archivo añadido, y al que hemos realizado varios commits con algunos cambios, solo nos queda crear el repositorio remoto correspondiente en GitHub y conectarlos.

Para ello entramos en nuestra cuenta de GitHub y pulsamos primero en el link "**Your repositories**" que se encuentra en el icono de la esquina superior izquierda. Por último, pulsamos sobre el botón verde "**New**" que está a la derecha:



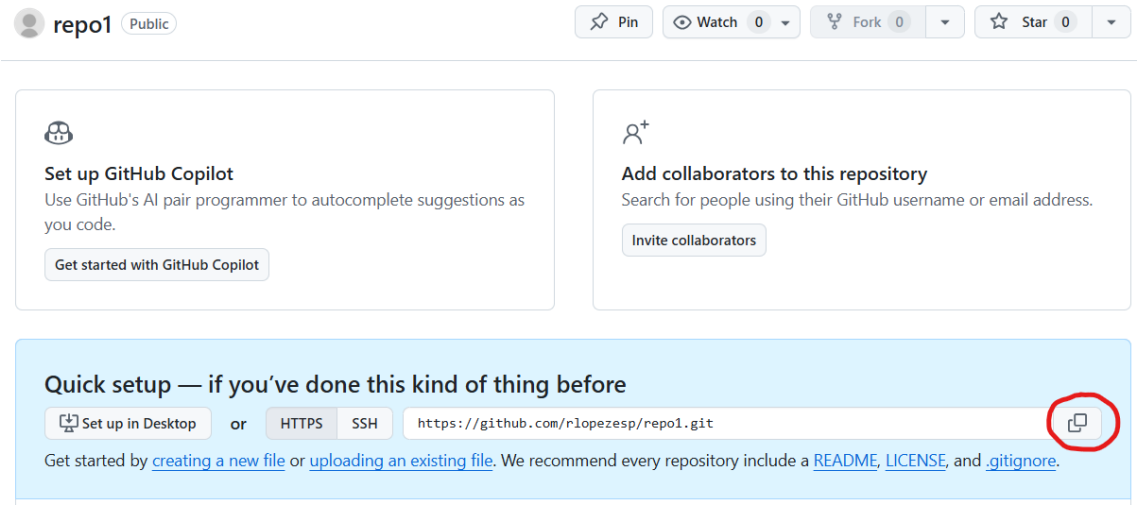
A continuación, en el campo "**Repository name**", escribimos el mismo nombre que el repositorio local, en mi caso "**repo1**". En "**Description**" ponemos una descripción, por ejemplo "**Repositorio para aprender a usar Git y GitHub**".

En la sección de configuración podemos dejar la visibilidad en pública o en privada. Y, por el momento, no añadimos ni *README*, ni archivo *gitignore*, ni *licencia*, simplemente le damos al botón "**Create repository**".



3.2. Enlazar un repositorio remoto a tu repositorio local

Primero vamos a copiar la URL del repositorio remoto de GitHub de la siguiente manera:



A continuación, abrimos la herramienta GitBash y entramos en el "working directory" de nuestro repositorio, en mi caso "repo1". Después, ejecutamos el siguiente comando con la URL que acabamos de copiar:

```
git remote add origin <url_tu_repo>
```

Por ejemplo:

```
git remote add origin https://github.com/rlopezesp/repo1.git
```

Para verificar que se ha creado una conexión remota al repositorio GitHub, es decir, que nuestro repositorio local está vinculado con el repositorio remoto, podemos ejecutar el comando "git remote -v":

```
$ git remote -v
origin  https://github.com/rlopezesp/repo1.git (fetch)
origin  https://github.com/rlopezesp/repo1.git (push)
```

Con esto ya tendríamos configurado nuestro repositorio local para poder hacer subidas (*git push*) al repositorio remoto, o traernos los cambios del repositorio remoto a local (*git pull*).

3.3. Enviar cambios al repositorio remoto

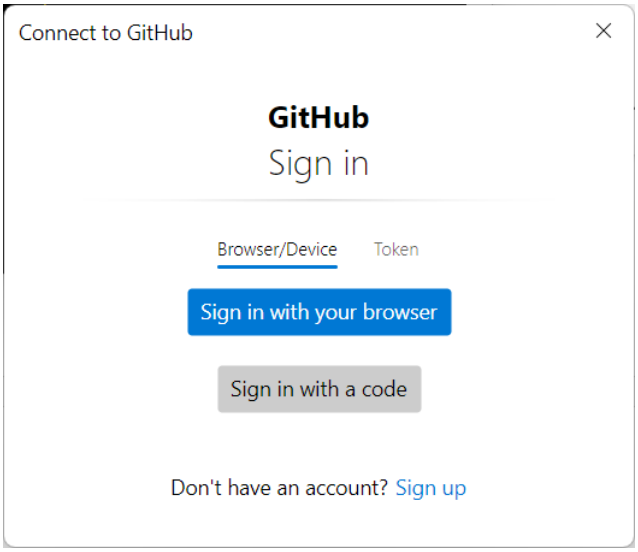
Ahora vamos a subir los ficheros que ya tenemos en nuestro repositorio local al repositorio remoto en GitHub. Esta acción incluirá todos los cambios registrados en nuestro repo local, es decir, todo el historial de commits, no solo la última versión.

Para ello debemos ejecutar el comando "**git push**" de la siguiente manera:

```
git push -u origin main
```

El **parámetro -u** será fundamental para crear una conexión entre ambos repos. Así, la próxima vez que quieras actualizar tu repositorio local podrás escribir solamente "git push", sin necesidad de hacer referencia al repo origin.

Al ejecutar el comando anterior, si trabajas desde un equipo Windows, se abrirá una ventana en la que te pedirán que hagas login con tu cuenta de GitHub. Pulsamos sobre el botón "Sign in with your browser" y **autorizamos el acceso**.



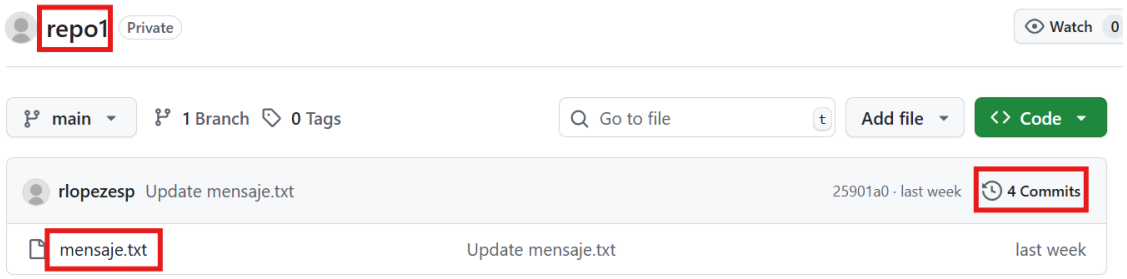
Si estás trabajando desde un equipo con Mac OS tendrás que autorizar el acceso escribiendo tu nombre de usuario y el código de un token. Para generar dicho token puedes consultar la siguiente página:

https://mgimond.github.io/Intro-to-git-GitHub/authenticating_with_github.html

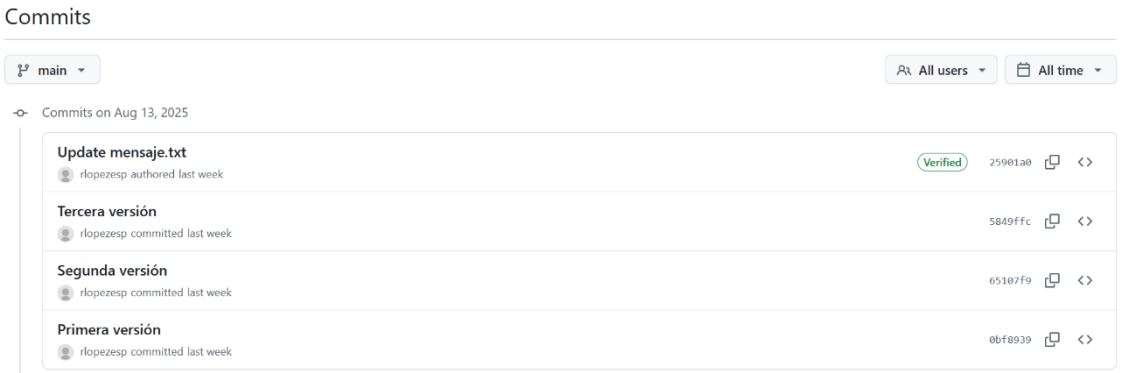
Una vez autorizado el acceso nos mostrará un mensaje similar al siguiente en el que nos informan de la subida de los ficheros del repositorio local al remoto:

```
$ git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (9/9), 676 bytes | 169.00 KiB/s, done.
Total 9 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/rlopezesp/repo1.git
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Si ahora vamos a la página de nuestro repo en GitHub y refrescamos veremos que los cambios ya se pueden ver en la rama main de nuestro repo remoto.



Si pulsas sobre el texto “4 Commits” puedes consultar el historial de commits de tu repositorio y comprobar que, efectivamente, se han subido todos los commits que realizaste en local:

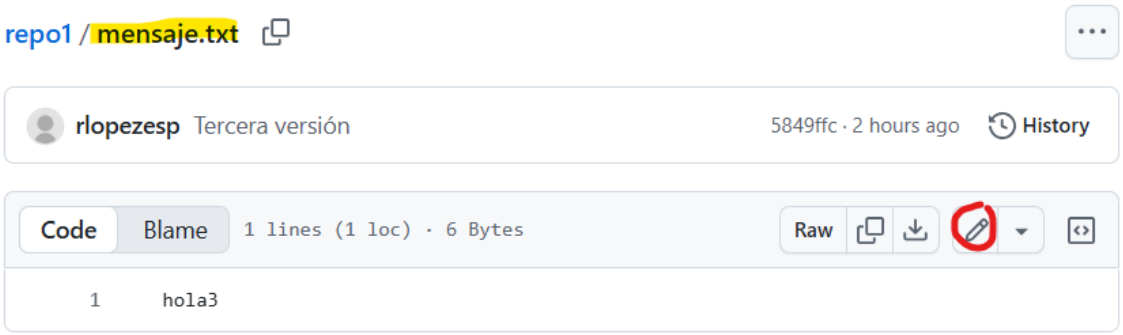


A partir de este momento, puedes realizar todos los cambios que quieras sobre el archivo "mensaje.txt" o crear nuevos archivos de diferente tipo (README, .gitignore, LICENSE, otros archivos con código, etc). Pero recuerda que estás trabajando sobre el repositorio local. Para subir dichos cambios al repositorio remoto tendrás que utilizar siempre el comando git push.

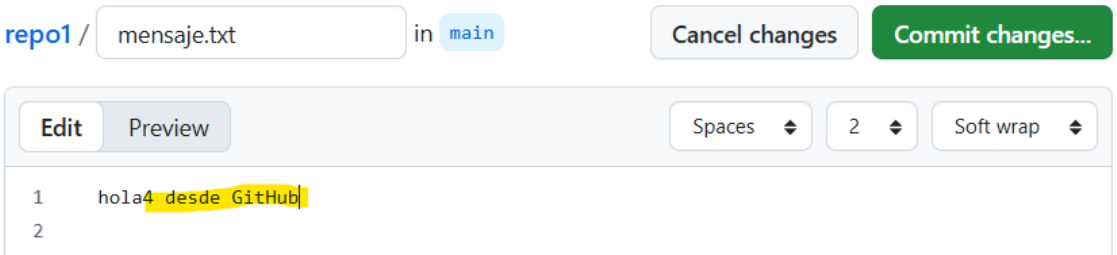
3.4. Recuperar cambios del repositorio remoto

Una vez que hemos conectado el repositorio local con el remoto y hemos subido los cambios al mismo estamos preparados para realizar la operación inversa: traer los últimos cambios del repositorio remoto al local.

Para ello, en primer lugar, vamos a realizar algún cambio en el repositorio remoto. Abre tu repositorio y pulsa sobre el archivo "mensaje.txt". A continuación, pulsa sobre el botón de edición:



Modifica el contenido del fichero y, después, pulsa el botón "**Commit changers...**"



Aparecerá una ventana en la que puedes modificar la descripción del commit y aceptar el mismo.

Ahora vamos a volver a Git Bash. En primer lugar, muestra el contenido de tu fichero “mensaje.txt”

```
cat mensaje.txt
```

Por último, para traer los cambios de tu repositorio local a tu repositorio remoto ejecutamos el comando **git pull** de la siguiente manera

```
$ git pull
Updating 5849ffc..25901a0
Fast-forward
mensaje.txt | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
```

Fíjate que estamos utilizando el comando corto “git pull”, en vez del comando completo “git pull origin main”. Esto es debido a que utilizamos el parámetro -u la primera vez que hicimos git push o git pull y git ya recuerda la asociación.

Podemos mostrar el contenido del fichero “mensaje.txt” para comprobar que se ha actualizado.

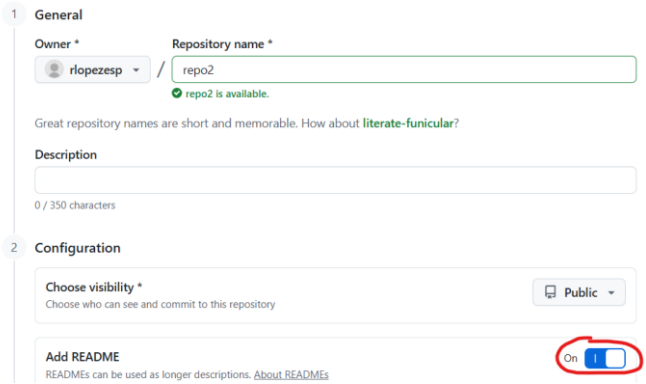
También podemos ver el historial de cambios para comprobar que se han traído los nuevos cambios del repositorio remoto.

3.5. Clonar un repositorio remoto

Ya hemos visto cómo conectar nuestros repositorios locales y remotos y traer cambios entre ellos. A continuación, vamos a ver cómo podemos clonar un repositorio remoto en nuestro ordenador.

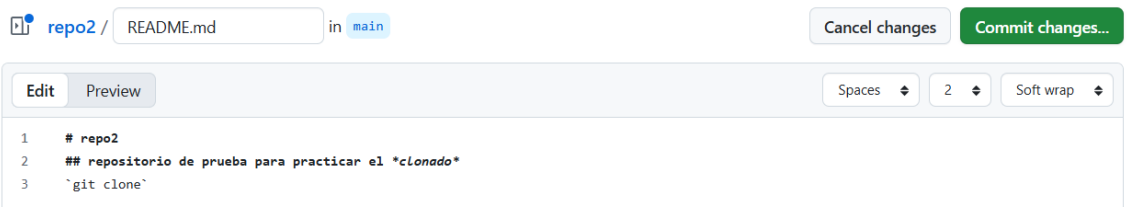
Clonar un repositorio remoto significa copiar todo el historial y los archivos de un repositorio remoto, que se encuentre por ejemplo en GitHub, en tu máquina local. Esto nos permite trabajar en el proyecto de manera local y después poder sincronizar tus cambios con el repositorio remoto cuando sea necesario. Es decir, conectará ambos repositorios (local y remoto) de manera automática. Así las operaciones de push y pull se podrán ejecutar directamente.

En primer lugar, vamos a crear un repositorio en nuestra cuenta de GitHub. Le llamaremos, por ejemplo "repo2". Pero, esta vez, vamos a crear un archivo README. Para ello pulsar sobre el botón correspondiente:

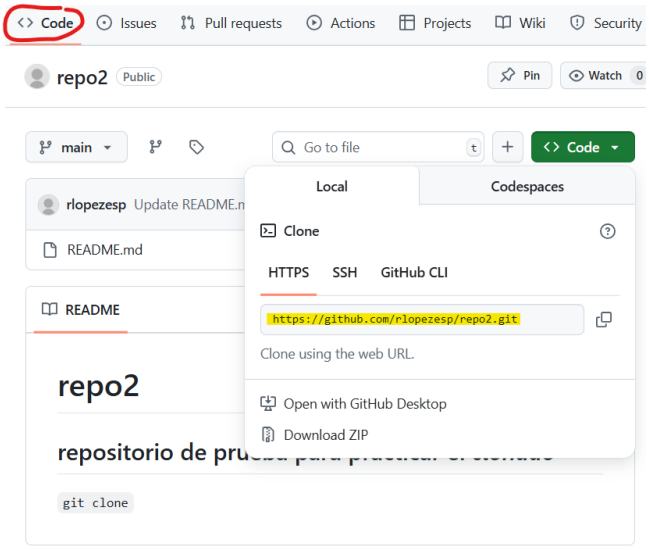


Al entrar en tu nuevo repo verás que se ha creado un fichero llamado "README.md". Este es un fichero Markdown que se utiliza muy frecuentemente para describir los proyectos de desarrollo.

Pulsamos sobre el botón de edición, modificamos el contenido de dicho fichero y realizamos un commit:



Ahora vamos a copiar la URL del repositorio. Para ello pulsamos sobre el botón <> Code, después, de nuevo sobre el botón <> Code verde y copiamos la URL que aparece:



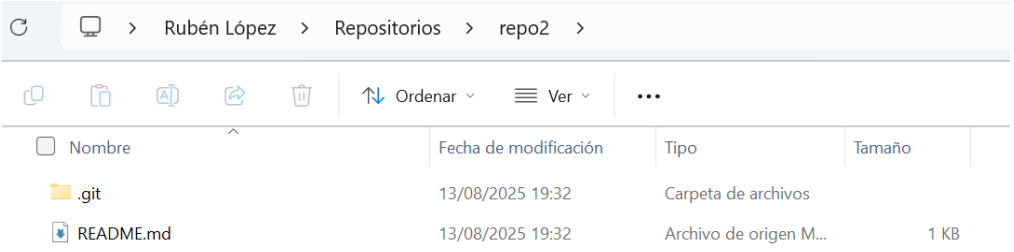
Para clonar el repositorio remoto en nuestro sistema abrimos la herramienta Git Bash y, en la carpeta que queramos crear el repositorio ejecutamos el siguiente comando:

```
git clone <url_repositorio>
```

Nos mostrará un mensaje parecido al siguiente:

```
$ git clone https://github.com/rlopezesp/repo2.git
Cloning into 'repo2'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (6/6), done.
```

Por último, podemos mostrar la lista de archivos y comprobamos que el nuevo repositorio se ha creado correctamente.



También podemos consultar el historial del repositorio y comprobar que se han descargado todos los commits del mismo, no solo el último:

```
$ git log
commit ee51d0e3fc49d183d2b340e4f0de7b1d8c8d0a9f (HEAD -> main,
origin/main, origin/HEAD)
Author: rlopezesp <rlopez.es@gmail.com>
Date:   Wed Aug 13 19:28:05 2025 +0200

    Update README.md

commit ca5cb3f09571af43b641bb4729635f42215f3c7c
Author: rlopezesp <rlopez.es@gmail.com>
Date:   Wed Aug 13 19:22:56 2025 +0200

    Initial commit
```

Es importante entender que, cuando clonas un repositorio, Git descarga toda la información de todas las ramas y commits del repositorio remoto. Sin embargo, **solo crea una rama local** para la rama por defecto (generalmente main).

Esto significa que, aunque tienes todos los datos en tu máquina, no verás las otras ramas si usas git branch. Para ver todas las ramas remotas que se descargaron, necesitas usar: git branch –r

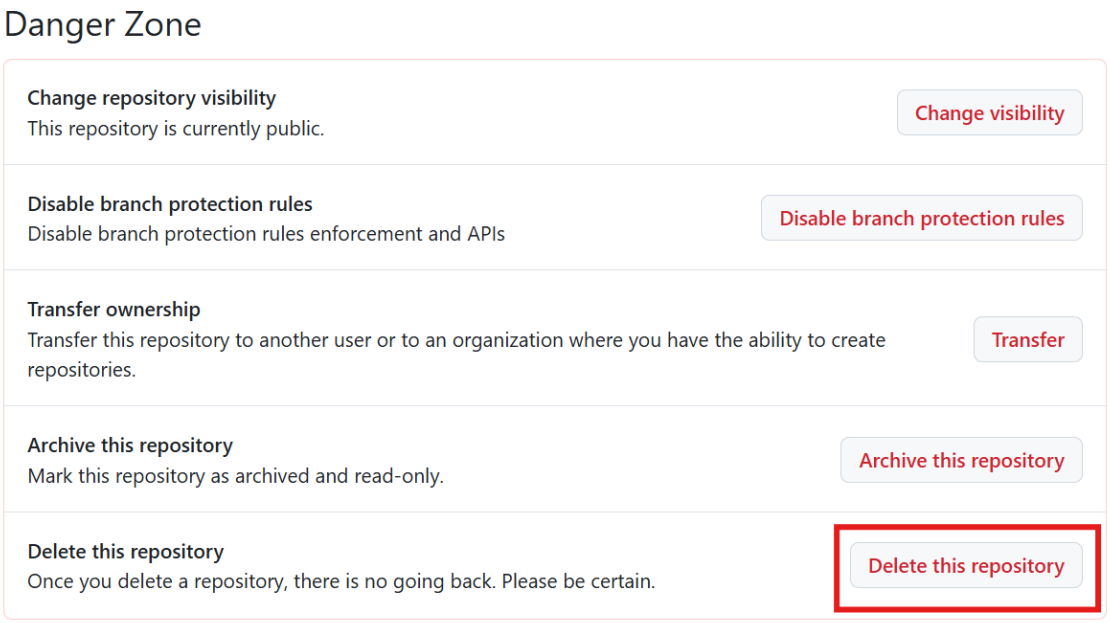
Después, para trabajar en otra rama, simplemente tienes que usar git checkout <nombre-de-la-rama>. Git creará la rama en local automáticamente y la configura para que rastree a la rama remota.

No obstante, trabajaremos con más ramas más adelante.

3.6. Borrar un repositorio remoto

Si no vas a volver a utilizar un repositorio remoto en GitHub puedes borrarlo. Pero cuidado, esta acción no se puede deshacer.

Para ello simplemente tienes que abrir tu repositorio en GitHub, pulsar sobre la pestaña "**Settings**" y bajar el scroll hasta encontrar la "Danger Zone". En ella encontrarás un botón que indica "Delete this repository".



4. Trabajando con Entornos de desarrollo

En este apartado vamos a ver cómo trabajar con un entorno de desarrollo (IDE) integrado con Git. Esto es recomendable ya que simplifica y visualiza el proceso de control de versiones.

Un IDE, por ejemplo, Visual Studio Code, ofrece una interfaz gráfica que te permite ver de manera intuitiva los cambios en tus archivos, realizar commits, gestionar ramas y resolver conflictos de fusión sin necesidad de memorizar y escribir comandos en la terminal. Esto hace que el flujo de trabajo con Git sea más eficiente, menos propenso a errores y más accesible, permitiéndote enfocarte en la codificación en lugar de en la gestión del historial de versiones.

No obstante, utilizar la línea de comandos sigue siendo útil ya que te ofrece control total y granular sobre las operaciones de Git, permite automatizar tareas complejas con scripts, y es universal, ya que funciona en cualquier sistema sin depender de un IDE específico.

4.1. IDEs más utilizados en Python

Un Entorno de Desarrollo Integrado (IDE, por sus siglas en inglés, Integrated Development Environment) es una aplicación de software que proporciona un conjunto de herramientas esenciales para que los programadores puedan escribir, probar y depurar código de manera más eficiente.

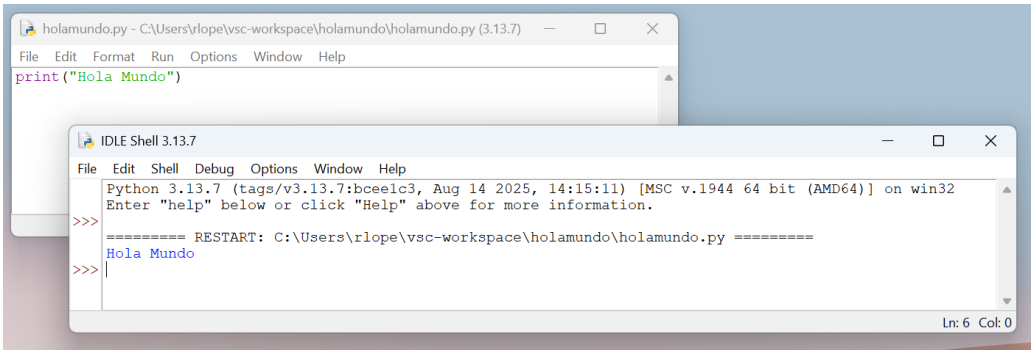
En lugar de tener que usar varios programas separados para cada tarea (un editor de texto para escribir, un compilador para traducir el código, etc.), un IDE integra todas estas herramientas en una única interfaz.

Las funciones más importantes que suelen incluir los IDE son:

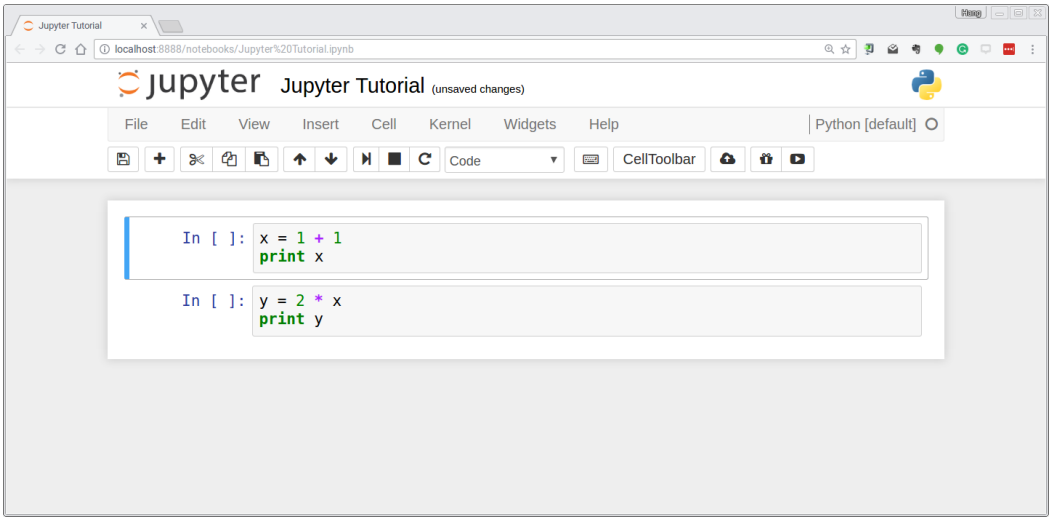
- Editor de código
- Compilador o intérprete
- Depurador
- Automatización del empaquetar el código o ejecución de pruebas

En Python existen varios entornos de desarrollo (IDEs) populares, pero si los ordenamos por facilidad de uso para alguien que empieza, teniendo en cuenta instalación, curva de aprendizaje y comunidad de apoyo, los más populares podrían ser:

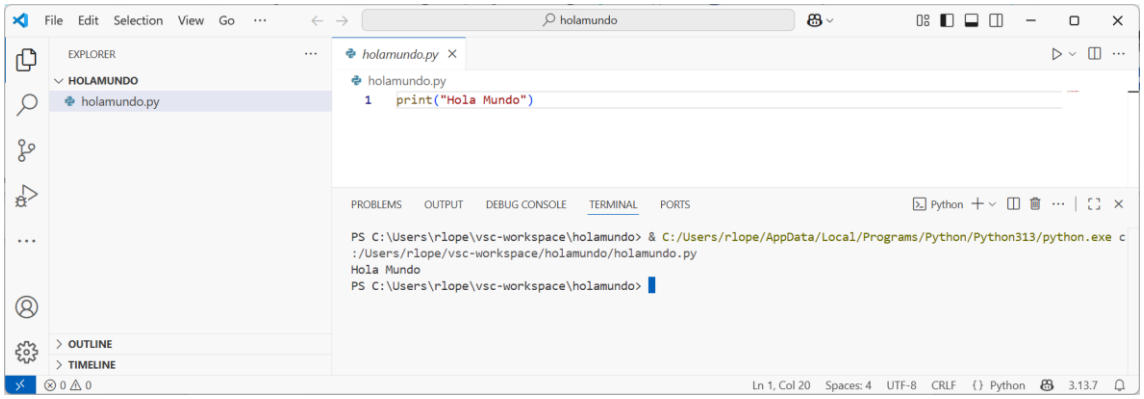
- **IDLE:** Es el entorno de desarrollo que se instala por defecto con Python. Está completamente escrito en Python, y su nombre es un homenaje a Eric Idle, uno de los miembros fundadores de Monty Python. Es muy sencillo y básico, ideal para quienes recién comienzan a aprender el lenguaje y no necesitan funcionalidades avanzadas.



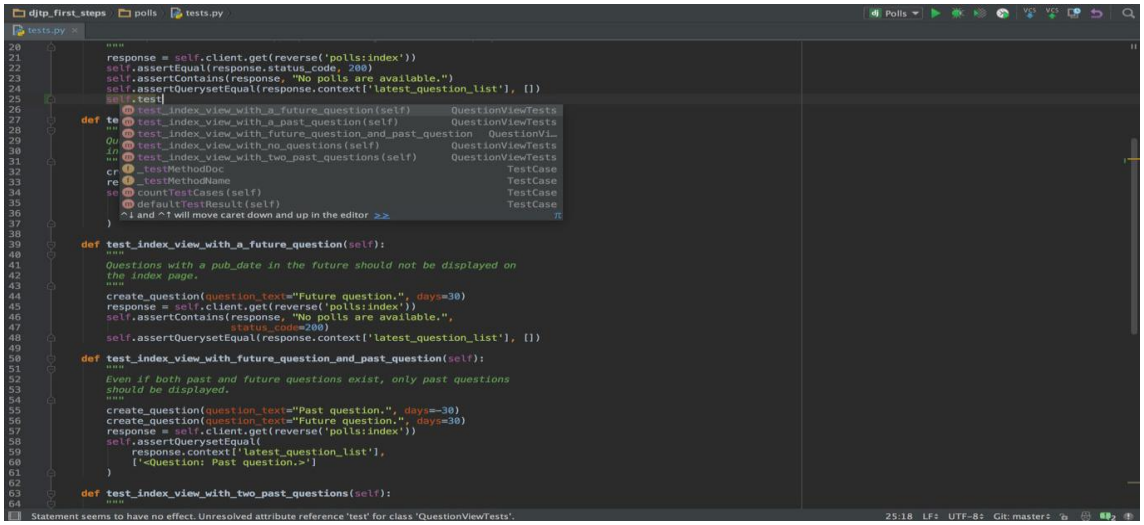
- **Jupyter Notebook y JupyterLab:** Son muy populares en la ciencia de datos y el aprendizaje automático. Permiten ejecutar código en "celdas" de forma interactiva y mezclarlo con texto y visualizaciones. Su naturaleza interactiva es muy amigable para la exploración de datos y el desarrollo de prototipos.



- **Visual Studio Code (VS Code):** Aunque es un editor de código, con la extensión de Python de Microsoft se convierte en un entorno de desarrollo muy potente y completo. Es muy popular por su flexibilidad, gran cantidad de extensiones y una interfaz intuitiva que se adapta bien a diferentes lenguajes.



- **PyCharm:** Es un IDE muy completo y potente, considerado el estándar profesional. Aunque tiene una versión gratuita (Community Edition), su gran cantidad de características, como el autocompletado inteligente, análisis de código y depuración avanzada, pueden ser abrumadoras para los principiantes, pero son extremadamente útiles para desarrolladores más experimentados.



4.2. Instalación de Python y Visual Studio Code

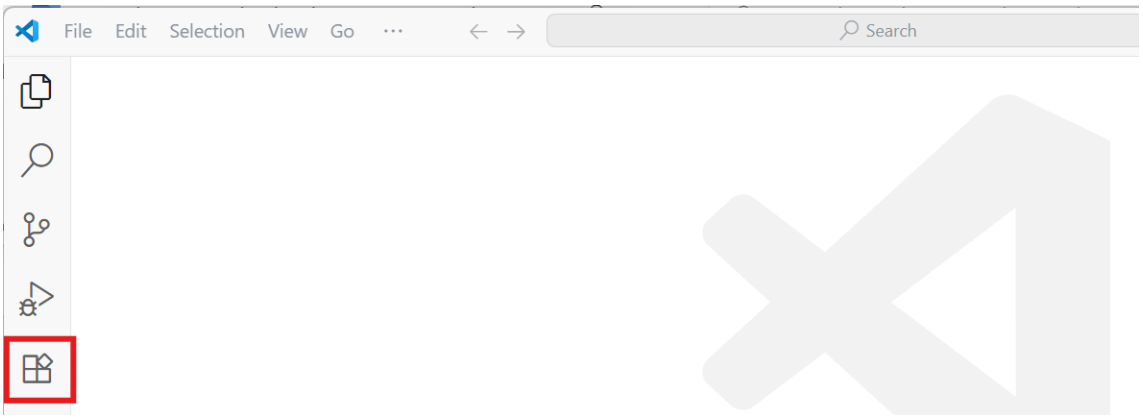
Para comenzar a usar Python y Visual Studio Code, primero debes instalarlos en tu sistema. Los pasos varían según el sistema operativo que estés utilizando. Descarga el instalador de Python y de Visual Studio Code de sus páginas oficiales y ejecuta ambos archivos:

https://www.python.org/downloads/

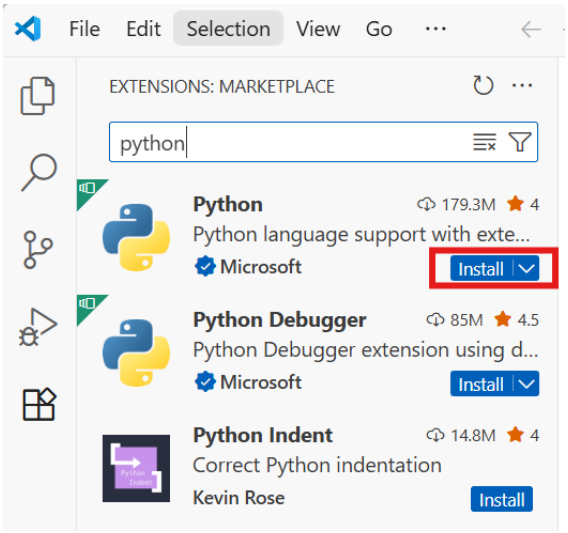
https://code.visualstudio.com/

Puedes consultar el "**ANEXO II. Instalación IDE**" para más detalles.

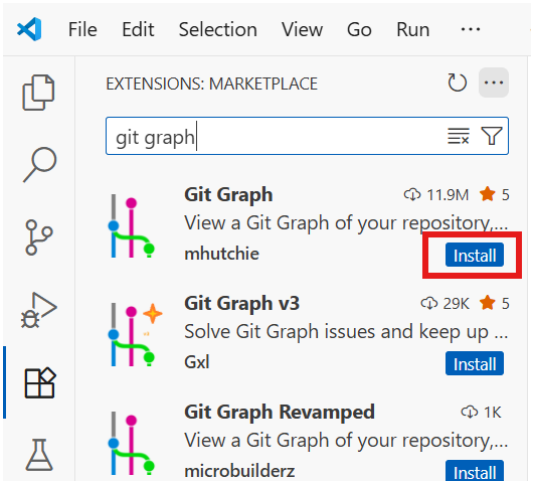
Una vez terminada la instalación, debemos añadir algunas extensiones al Entorno de trabajo (IDE). Para ello pulsamos sobre el icono "**Extensions**" en el menú de la izquierda:



A continuación, en el buscador, escribimos "**python**" e instalamos la primera extensión que nos ofrece. Esta extensión nos permite trabajar con proyectos escritos en lenguaje Python dentro de Visual Studio Code. Para que esta integración funcione correctamente previamente debemos haber instalado Python en nuestro sistema.



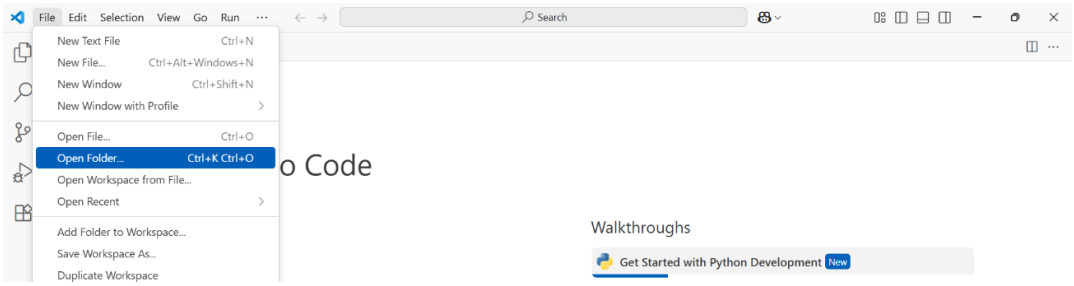
Y, por último, por ahora, vamos a descargar otra extensión muy útil para trabajar con ramas. Para ello escribimos "**git graph**" en el buscamos e instalamos la primera extensión que nos ofrece:



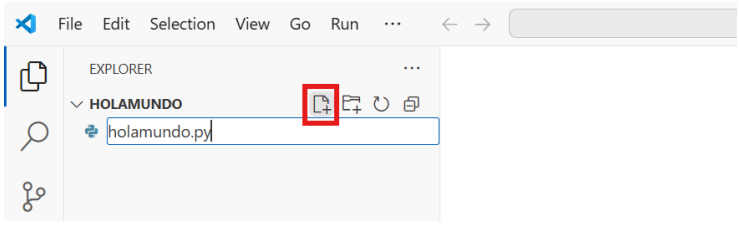
4.3. Crear un proyecto de Python

Para crear un nuevo proyecto de Python, en primer lugar, crearemos en nuestro equipo una carpeta donde ubicar los archivos de nuestro proyecto.

A continuación, abriremos dicha carpeta utilizando Visual Studio Code. Para ello existe varias posibilidades. Una de ellas sería abrir el IDE Visual Studio Code, entrar en el menú "**File**" y después escoger la opción "**Open Folder**".



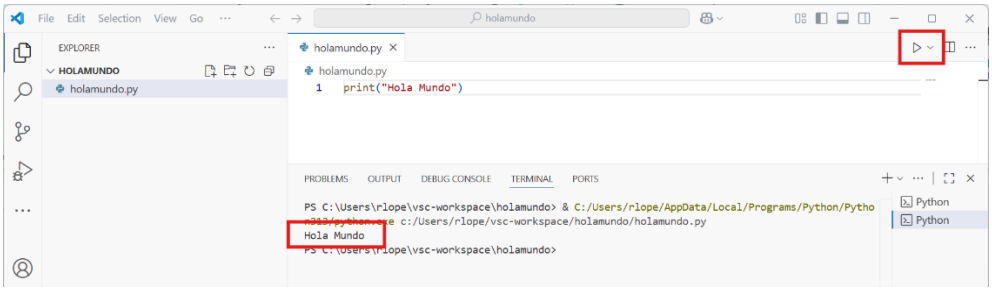
Una vez abierta la carpeta podemos crear un primer archivo de Python pulsando sobre el botón "**New File**". Después habrá que escribir el nombre del archivo y pulsar la tecla Enter.



Ahora podemos escribir el código que queramos, por ejemplo:

```
print("Hola Mundo")
```

Por último, para **ejecutar el código**, pulsaremos sobre el botón play, situado en la esquina superior derecha. Podemos ver el resultado de la ejecución en el Terminal de la parte inferior:

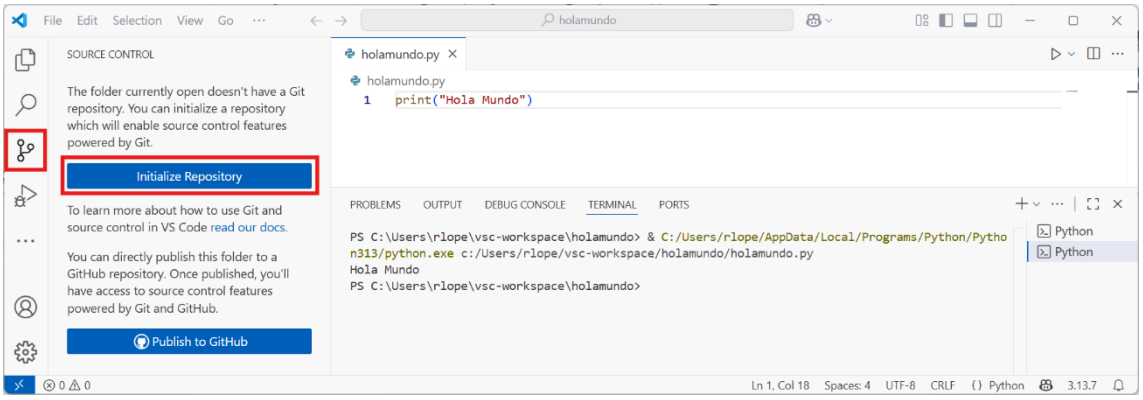


Cuando trabajamos con proyectos Python en Visual Studio Code es habitual utilizar **Entornos Virtuales**. Un entorno virtual es un directorio que contiene una instalación de Python y una copia de las bibliotecas necesarias para un proyecto específico. Su propósito es aislar las dependencias de tu proyecto del resto de la máquina, evitando conflictos entre versiones de paquetes de diferentes proyectos. No obstante, nosotros no los utilizaremos ya que vamos a realizar proyectos muy sencillos que no importan ninguna librería.

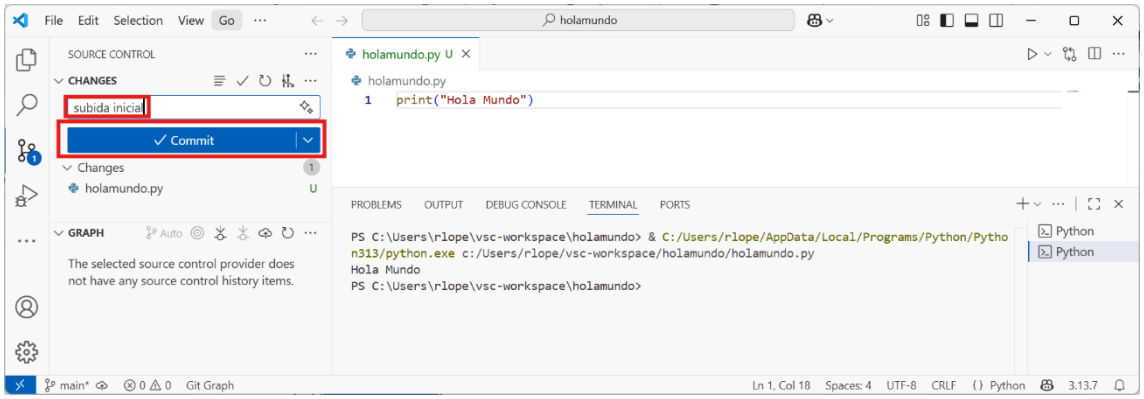
4.4. Crear un repositorio local

Una vez creado nuestro proyecto de Python podemos convertirlo en un repositorio para, de esta manera, poder disfrutar de las ventajas que los repositorios nos ofrecen: recuperar versiones anteriores, realizar un seguimiento de los cambios, etc.

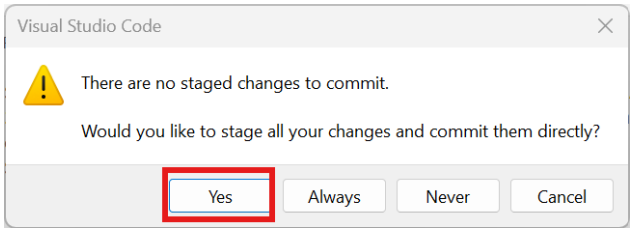
Para ello vamos a escoger la opción "**Source Control**" del menú de la izquierda y, después, pulsaremos sobre el botón "**Initialize Repository**". Esto creará un repositorio en la carpeta actual, sería equivalente a ejecutar el comando git init en nuestro Terminal, desde la carpeta del proyecto.



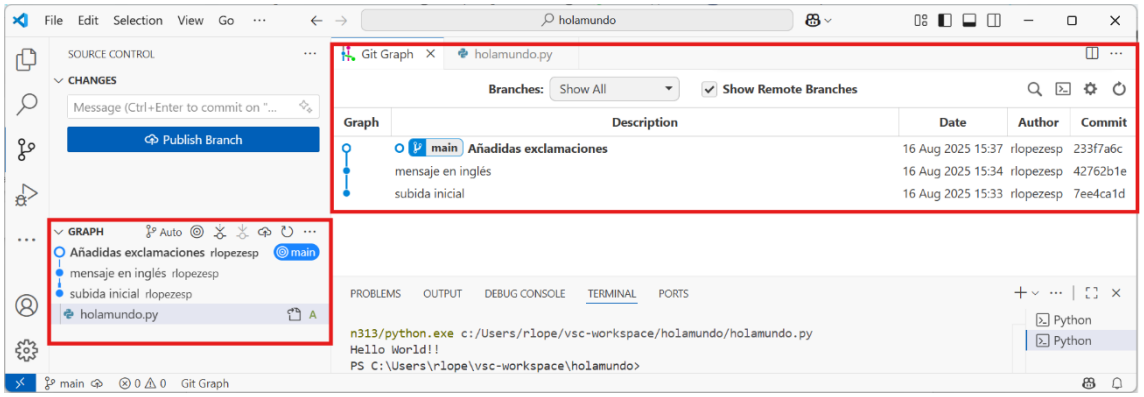
A continuación, escribimos una **descripción** para el primer commit y pulsamos sobre el botón "**Commit**". Esto sería equivalente a ejectuar el comando git commit –m "subida inicial":



Nos preguntará si deseamos añadir los ficheros modificados al staging area. Pulsamos sobre el botón "**Yes**" o “Always”. Esta acción sería equivalente a ejectuar el comando git add .



Podemos realizar más cambios o sus commits correspondientes y ver el historial de cambios, tanto en GRAPH como en Git Graph:

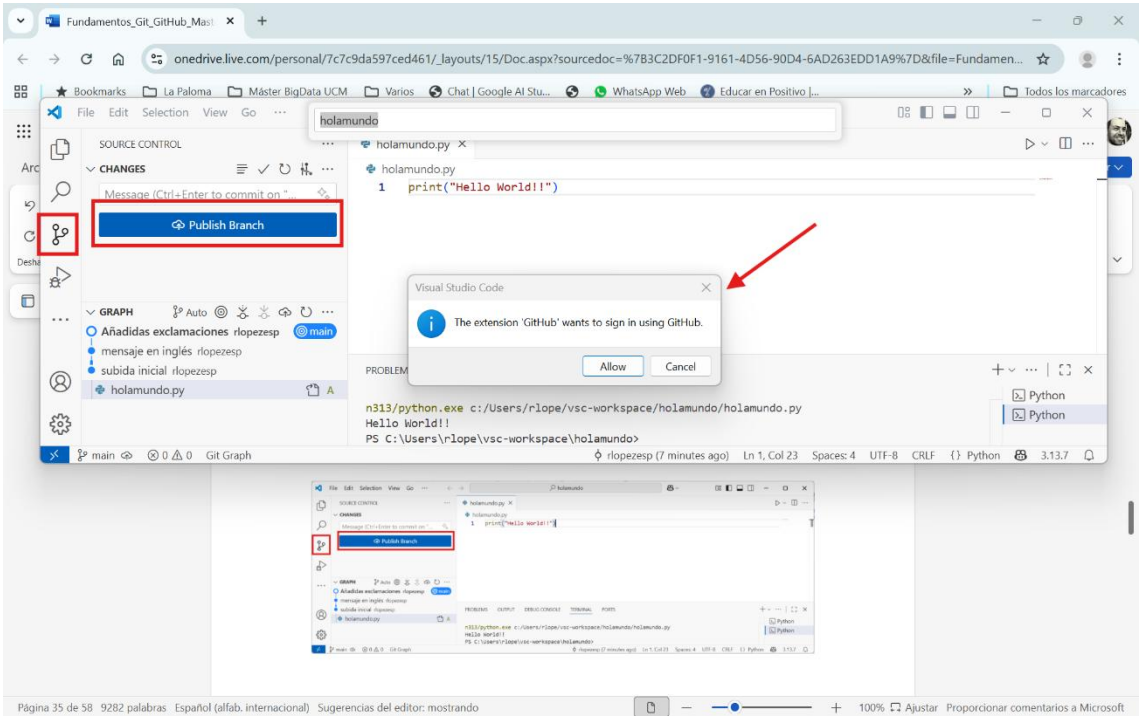


4.5. Crear un repositorio remoto

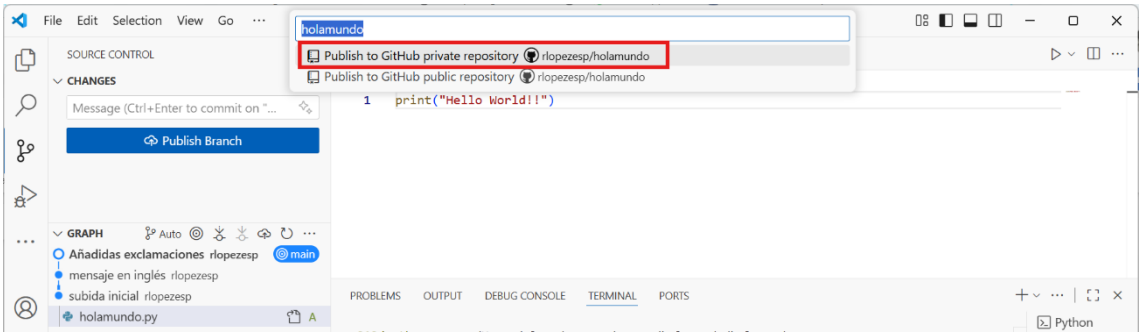
Ya hemos creado nuestro primer proyecto de Python, lo hemos ejecutado y hemos creado un repositorio local, de manera que podamos disponer de un historial de cambios. Todo ello utilizando Visual Studio Code.

Ahora vamos a crear un repositorio remoto, asociado a nuestro repositorio local, para que, de esta manera, podamos subir nuestro proyecto a GitHub y trabajar con él de manera colaborativa.

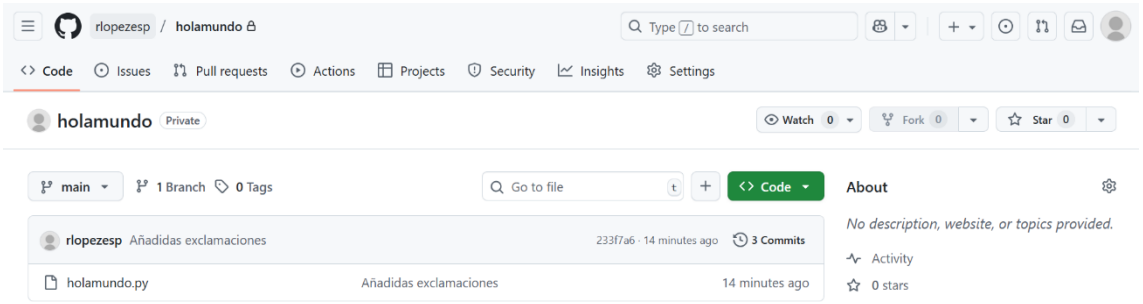
Para ello vamos a escoger la opción "**Source Control**" del menú de la izquierda y, después, pulsaremos sobre el botón "**Publish Branch"**. Al pulsar el botón, si es la primera vez que lo hacemos, nos pedirá que hagamos login en nuestra cuenta de GitHub. Pulsamos "**Allow**", escribimos nuestro usuario y contraseña en la ventana del navegador que se habrá abierto y autorizamos el acceso.



En la parte superior se mostrará un mensaje en el que nos preguntará el **nombre** del repositorio remoto, por defecto el mismo nombre que el repositorio local y la **privacidad**. Pulsamos sobre el tipo de repositorio que queramos, por ejemplo, privado.

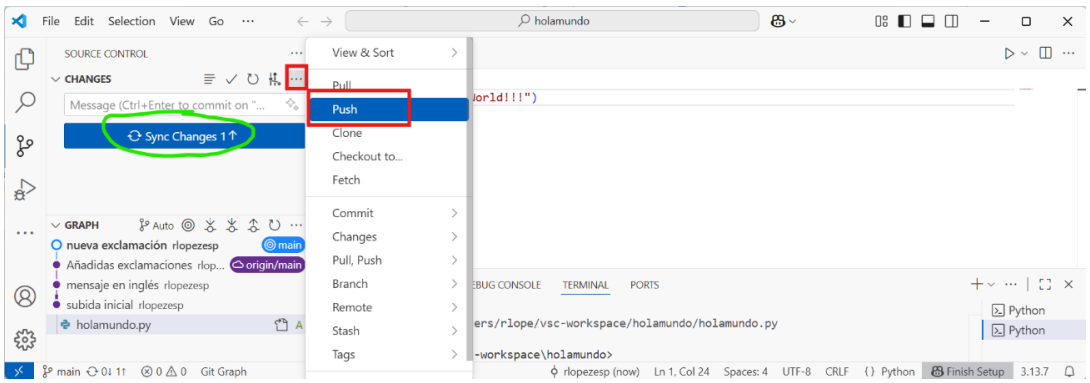


Pasados unos segundos podremos ver nuestro repositorio remoto. Para ello entramos en nuestra cuenta de GitHub y, en tus repositorios, podrás ver tu nuevo repositorio, “holamundo” en mi caso:

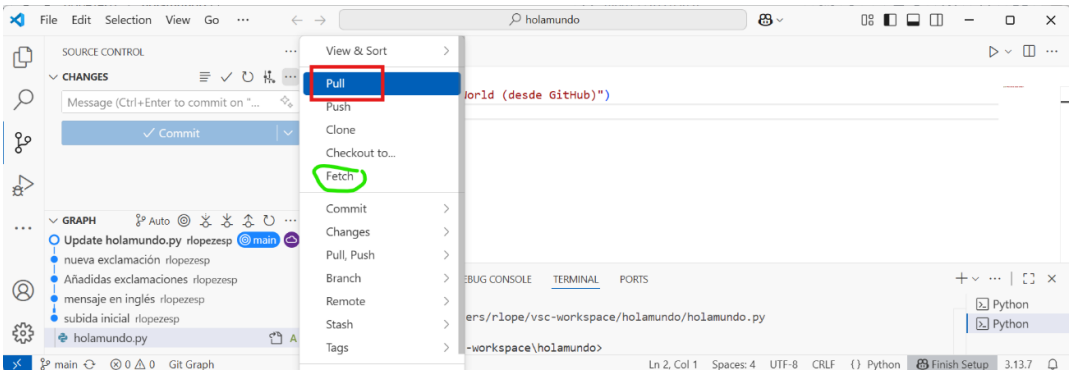


A partir de este momento podrás subir tus cambios al repositorio remoto y recuperar todas las novedades del mismo.

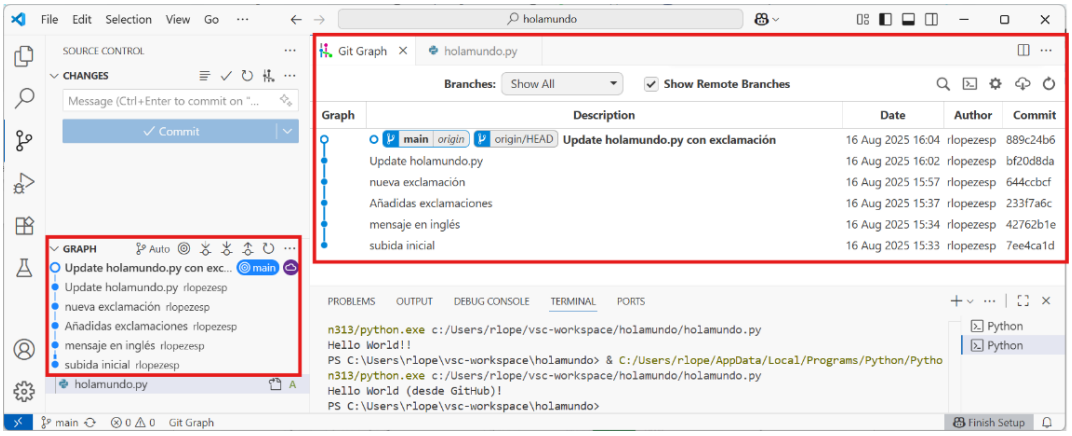
- **Subir cambios al repositorio remoto**: En primer lugar, tendremos que escribir algún cambio en el código y realizar un “**Commit**” del mismo. A continuación, pulsaremos sobre el botón con tres puntos y, a continuación, sobre la opción “**Push**”. También podríamos pulsar el botón “Sync Changes”. El entorno detectará si tiene que subir los cambios o, por el contrario, recuperarlos del repositorio remoto.



- **Traer cambios del repositorio remoto**: En este caso, en primer lugar tendremos que realizar algún cambio en el repositorio remoto de GitHub. Después, pulsaremos el botón con tres puntos y, a continuación, sobre la opción "**Pull**". También podríamos pulsar sobre la opción "**Fetch**" que comprueba si hay cambios disponibles en el repositorio remoto (pero no los trae). Después habría que pulsar sobre el botón "Sync Changes".



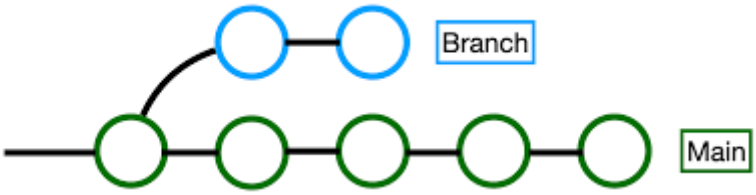
Recodar que podemos ver el historial de cambios desde la opción "**GRAPH**" o desde la extensión "**Git Graph**".



5. Trabajando con Ramas

5.1. ¿Qué son las ramas (branches) en Git?

Las ramas son líneas de desarrollo independientes en un repositorio Git. Te permiten trabajar en características nuevas o correcciones de errores sin afectar la rama principal (que generalmente llamada “**main**”).



En Git, una rama representa una versión independiente del repositorio, que se crea a partir de un commit ya existente. Es decir, cuando creas una rama, Git toma una "instantánea" del estado actual de tu proyecto en la rama desde la que la creaste. A partir de ese momento, tu nueva rama y la original pueden evolucionar de forma totalmente separada, aunque lo habitual será que la nueva rama se termine fusionando con la rama original.

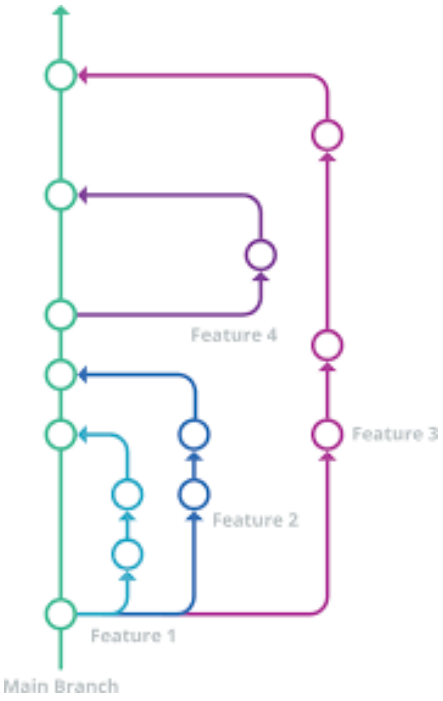
Las ramas son extremadamente útiles para la colaboración y la organización del trabajo de un equipo de desarrollo. Gracias a ellas:

- **Puedes trabajar en paralelo**: Varios desarrolladores pueden trabajar en diferentes funciones al mismo tiempo sin interferir entre sí.
- **Se aísla el trabajo**: Los cambios en una rama experimental o inestable no afectan al código principal y funcional de tu proyecto.
- **Es más seguro**: Si algo sale mal en una nueva rama, simplemente la puedes borrar sin haber afectado tu rama principal.
- **Se simplifica la revisión de código**: Puedes pedir a tus compañeros que revisen los cambios en tu rama antes de fusionarlos, lo que mejora la calidad del código.

Un ejemplo del uso en trabajo paralelo son las ramas para el desarrollo de una nueva funcionalidad o **ramas "feature"**. El flujo habitual de trabajo para este tipo de ramas es el siguiente:

- Creación de una rama: Esto crea un puntero al último commit de la rama principal (generalmente main), que permite la evolución del proyecto, de forma paralela, desde ese punto.
- Cambio a una rama: Para empezar a trabajar en ella, debes "cambiarte" a esa rama
- Trabajo en la rama: Aquí haces tus cambios, los agregas al staging area y haces tus commits como de costumbre. Estos commits solo existen en esta nueva rama.
- Combinar ramas: Una vez que tu nueva característica está completa y probada, puedes fusionar (merge) tu rama de vuelta a la rama principal (main). Esto integra los cambios que hiciste en la rama secundaria a la rama principal, manteniendo un historial limpio y seguro.

La siguiente imagen muestra, de forma gráfica, el trabajo con una rama principal (main) y ramas para cada una de las características que se quieren desarrollar. Como se ve, esta forma de trabajar facilita la integración del trabajo de todos los miembros del equipo de desarrollo, garantizado la integridad del producto resultante.



5.2. Creación de ramas

En primer lugar, vamos a listar qué ramas tiene nuestro repositorio y vamos a comprobar en qué rama nos encontramos, con el comando "**git branch**":

```
git branch
* main
```

Nos indica que estamos en la rama "**main**", que es la rama principal. Aunque en todos los ejemplos anteriores hemos trabajado sobre la rama main, esto no siempre será adecuado. La rama principal suele estar reservada para albergar versiones estables y completas de nuestro proyecto o código.

Siguiendo las buenas prácticas, vamos a crear una nueva rama a partir de la rama main que tenemos en este momento. A esta **nueva rama** la vamos a llamar "feature/nuevosaludo":

```
git branch feature/saludo
```

Ahora **listamos** de nuevo las ramas que tenemos en nuestro repositorio local:

```
git branch
  feature/saludo
* main
```

El asterisco * nos indica en qué rama nos encontramos de las existentes. Como se puede ver, aunque ahora existe la rama saludo, seguimos ubicados en la rama main. Para **cambiarnos** de la rama "main" a "feature/saludo" podemos usar el comando "git switch" indicando el nombre de la rama a la que queremos movernos:

```
git switch feature/saludo
Switched to branch 'feature/saludo'
```

Volvamos a listar las ramas y veamos en cual nos encontramos ahora:

```
git branch
* feature/saludo
  main
```

También se podría crear la rama y cambiarnos a ella con una sola línea. Para ello podemos utilizar el siguiente comando:

```
git switch -c feature/saludo
```

Desde este momento podemos tener la tranquilidad de que si hacemos algunos cambios erróneos o nos equivocamos y "rompemos" algo, el código original estará a salvo en la rama **main**, todo lo que hagamos lo haremos en la rama **feature/saludo**. Un símil podría ser escribir sobre una fotocopia de un documento importante.

Por lo tanto, a partir de este momento trabajaremos en nuestra nueva rama como aprendimos a hacer en los apartados anteriores:

- Podemos modificar, borrar o crear nuevos archivos
- Después añadiremos los cambios al staging area
- Por último, haremos uno o varios commits para subir los cambios al repositorio local.

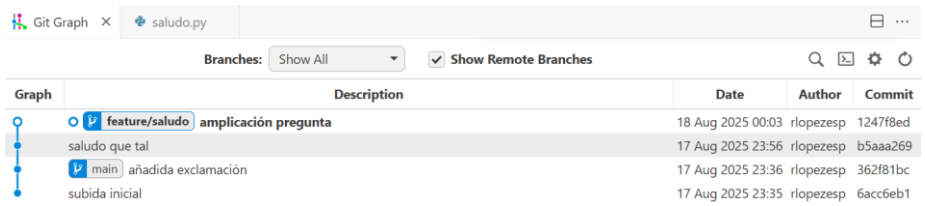
Si deseamos consultar el **historial de cambios**, como vimos anteriormente podemos seguir utilizando el siguiente comando:

```
git log
```

No obstante, ahora que estamos trabajando con ramas, nos puede resultar más útil utilizar el siguiente comando, ya que nos muestra la información en forma de árbol

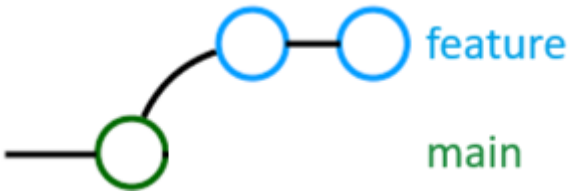
```
git log --graph
```

También podemos utilizar la extensión "Git graph" de Visual Studio Code, ya que nos muestra la misma información, pero de una manera más cómoda de visualizar, permitiéndonos incluso interactuar con las ramas, como veremos más adelante.



5.3. Fusionar ramas (merge)

En este momento tenemos dos ramas, la principal llamada **main** y otra donde trabajamos habitualmente llamada **feature/saludo**. En esta última hemos añadido un saludo al fichero Python.



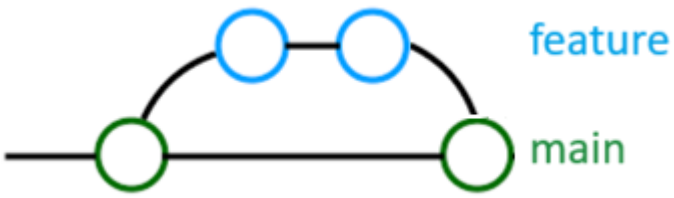
Antes de fusionar las ramas debemos mover a la rama destino, es decir, a la rama en la que queremos volcar el trabajo realizado en otra rama de trabajo. En nuestro caso nos moveremos a la rama **main**:

```
git switch main
Switched to branch 'main' Cambiado a rama 'main'
```

Ahora ejecutamos el comando "**git merge**" indicando el nombre de la rama donde se encuentra el trabajo que nos queremos traer o volcar en la rama en la que nos encontramos. En nuestro caso el comando sería el siguiente:

```
$ git merge feature/saludo
Updating 362f81b..1247f8e
Fast-forward
saludo.py | 3 ++-
1 file changed, 2 insertions(+), 1 deletion(-)
```

Este comando fusionará la rama feature con la rama principal (main). Gráficamente, el resultado de dicha fusión podría ser algo parecido a esto:

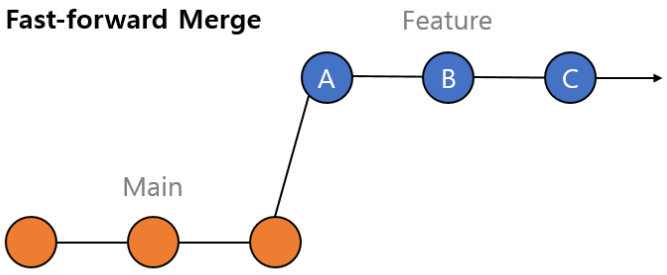


Cabe destacar que existen diferentes tipos de fusión (merge) que Git utilizará según la situación en la que se encuentran las ramas o los parámetros utilizados al realizar dicha fusión.

Las tres **fusiones más comunes** son:

1. **Fast-Forward**: Este tipo de fusión ocurre cuando la rama de destino (por ejemplo, main) no ha tenido nuevos commits desde que se creó la rama de origen (por ejemplo, feature). En esta situación, Git simplemente mueve el puntero de la rama main hacia adelante para que apunte al último commit de la rama feature.

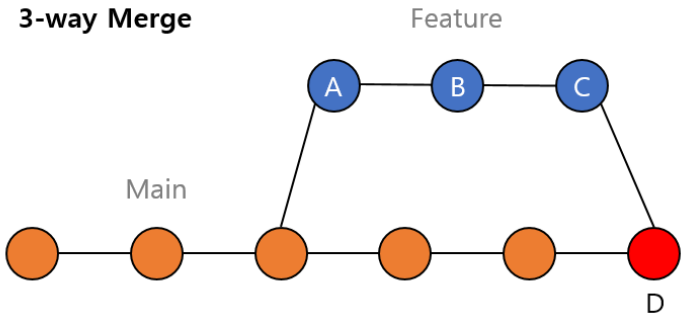
   Características:
   - No crea un nuevo commit de fusión. El historial permanece lineal.
   - Es la estrategia predeterminada cuando es posible.
   - Mantiene el historial limpio y fácil de seguir, ya que no hay commits extra de fusión.
   - No hay conflictos de fusión porque no hay cambios divergentes.



2. **No Fast-Forward o Three-Way**: Es el tipo de fusión predeterminado cuando los historiales de las dos ramas han divergido (es decir, ambas ramas han tenido nuevos commits desde que se bifurcaron). Git toma el último commit de la rama main, el último commit de la rama feature, y el antepasado común entre ambas. A partir de estos tres puntos, Git crea un nuevo commit especial llamado "merge commit".

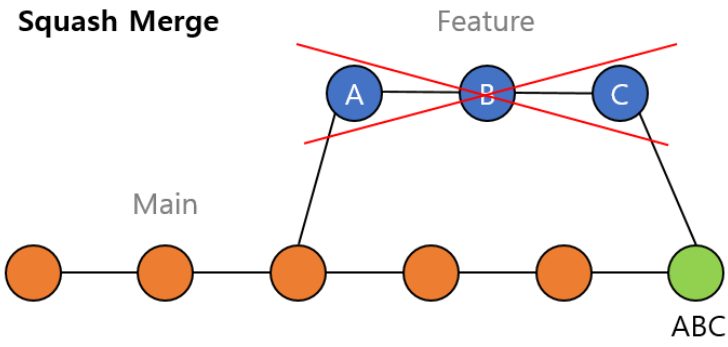
   Características:
   - Crea un nuevo commit de fusión. Este commit tiene dos padres y registra la fusión.
   - Preserva el historial de la rama de desarrollo.
   - Es la estrategia más común y segura para flujos de trabajo colaborativos.

- Puede requerir la resolución manual de conflictos si hay cambios en las mismas líneas de los mismos archivos.



3. **Squash**: Esta estrategia es una forma de consolidar múltiples commits de una rama en un solo commit antes de fusionarlos en la rama de destino. En lugar de traer todos los commits de la rama feature, el merge --squash combina todos esos cambios en un solo commit, que luego se aplica a la rama main como un commit normal.

   Características:
   - Crea un único commit en la rama de destino. El historial de la rama de origen no se mantiene.
   - No es una verdadera fusión porque no tiene dos padres; parece un commit normal.
   - Ideal para mantener un historial de commits lineal y limpio en la rama principal.
   - Se utiliza a menudo en plataformas como GitHub o GitLab al cerrar una pull request, ya que permite a los desarrolladores mantener su historial de commits personal sin importarlo a la rama main.



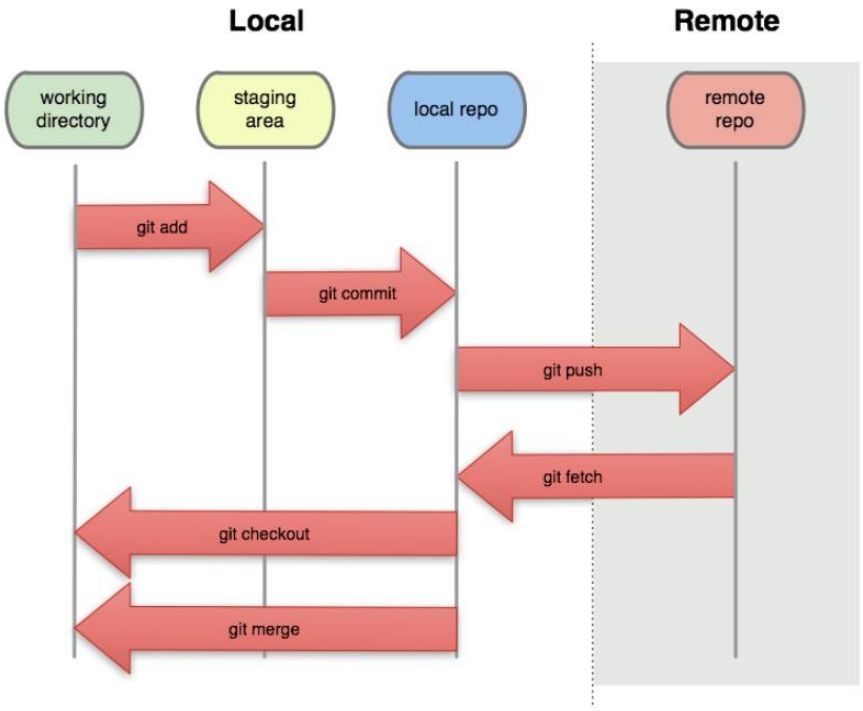
Por último, indicar que, una vez que has fusionado tu rama feature en la rama principal (main) y el código funciona correctamente, la rama feature ya no es necesaria y se suele **borrar**. Para ello se pueden utilizar los siguientes comandos:

```
git branch -d <nombre_rama>
```

Si la rama había sido subida al repositorio remoto además haremos:

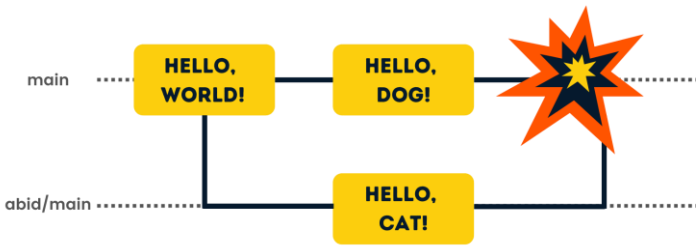
```
git push origin -delete <nombre_rama>
```

En los siguientes apartados empezaremos a utilizar tu entorno de desarrollo para trabajar con Git, por lo que utilizaremos menos los comandos por terminal. Así que, a modo de resumen, en el siguiente esquema puedes ver los comandos básicos de Git para trabajar con repositorios locales y remotos:



5.4. Resolviendo conflictos

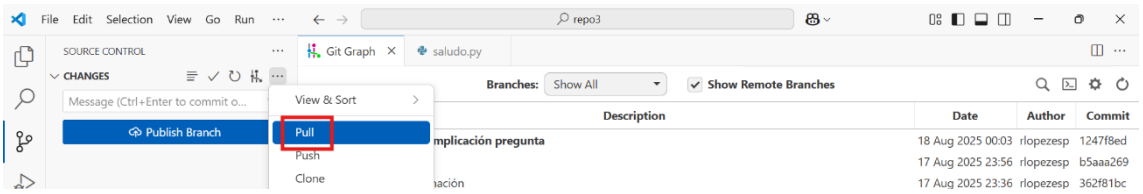
En ocasiones, al realizar una fusión de una rama sobre otra se pueden producir conflictos que Git no puede resolver y, por lo tanto, solicita al usuario que los resuelva. Estos conflictos se producen, por ejemplo, cuando se han realizado **cambios en las mismas líneas de código** tanto en la rama principal como en la rama donde se desarrollaba una funcionalidad.



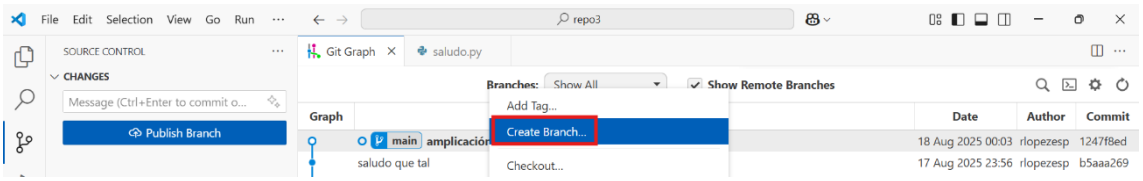
En este apartado vamos a ver cómo se pueden resolver dichos conflictos. Para ello nos apoyaremos en las herramientas que nuestro entorno de desarrollo (Visual Studio Code) nos proporciona.

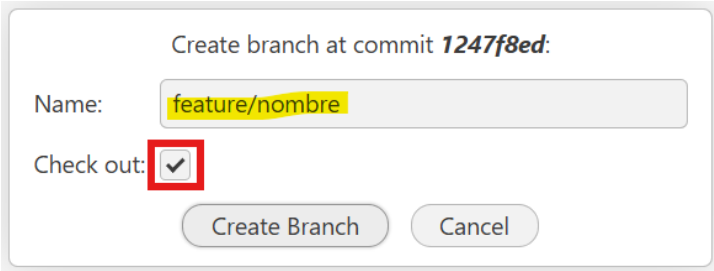
Seguiremos el siguiente flujo de trabajo:

1. Si tu repositorio local está conectado con un repositorio remoto asegúrate de que tu **rama principal esté actualizada**. Para ello puedes utilizar los comandos git checkout main y git pull. También puedes utilizar tu entorno de trabajo (IDE):

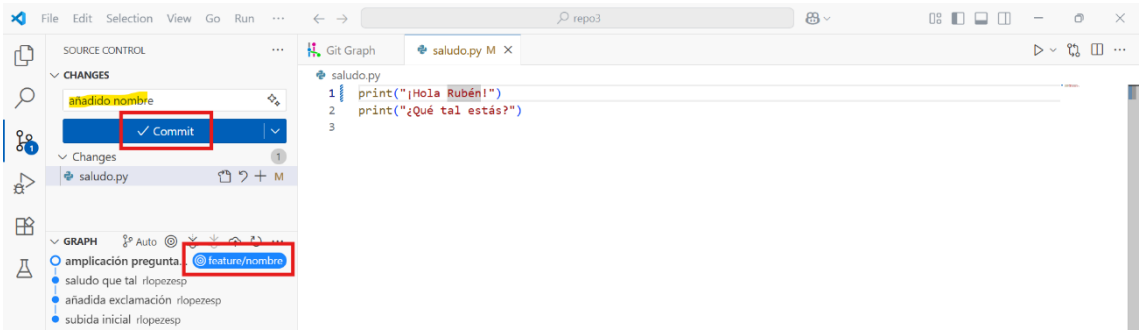


2. **Crea la nueva rama** para tu feature y asegúrate de estar ubicado en ella.

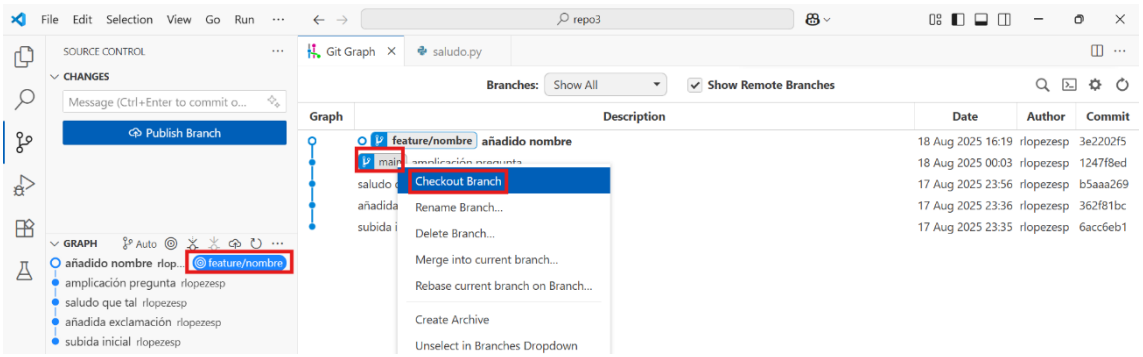


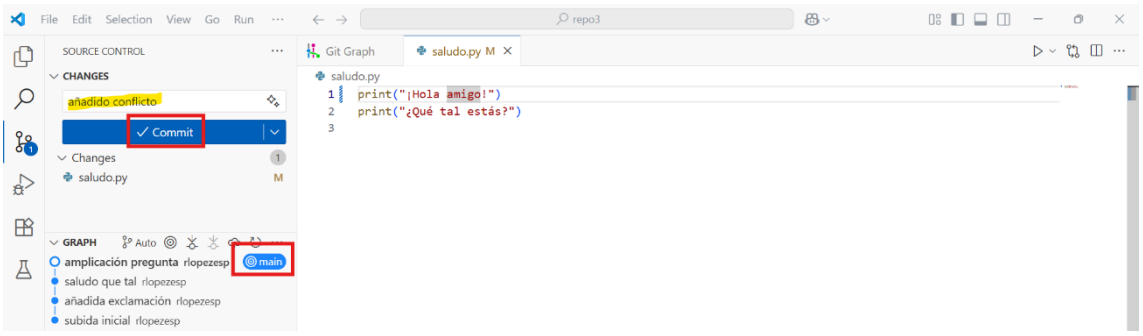


3. Realiza los **cambios** necesarios y haz commit en la nueva rama feature.



4. Antes de fusionar los cambios vamos a **provocar un conflicto**. Para ello vamos a volver a la rama "**main**" y generar algún cambio en la misma línea de código que hemos modificado en la rama feature.

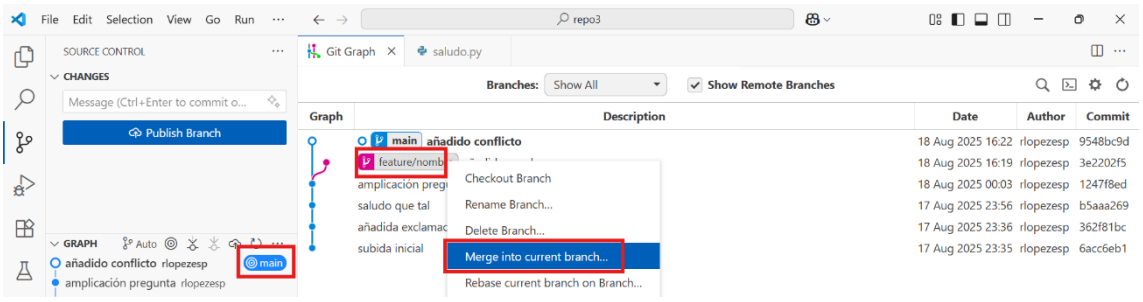


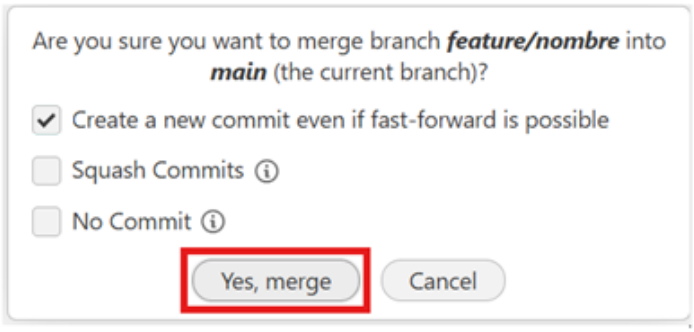


Si consultamos el historial en la extensión “Git graph” podemos comprobar que hay cambios en la rama principal (main) que están “por delante” de los cambios de la rama feature, es decir que son más reciente y no estaban en main cuando se creó la rama feature a partir de ella.

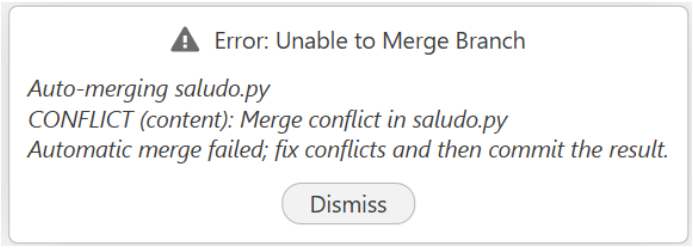
Graph	Description	Date	Author	Commit
	main añadido conflicto	18 Aug 2025 16:22	rlopezesp	9548bc9d
	feature/nombre añadido nombre	18 Aug 2025 16:19	rlopezesp	3e2202f5
	amplicación pregunta	18 Aug 2025 00:03	rlopezesp	1247f8ed
	saludo que tal	17 Aug 2025 23:56	rlopezesp	b5aaa269
	añadida exclamación	17 Aug 2025 23:36	rlopezesp	362f81bc
	subida inicial	17 Aug 2025 23:35	rlopezesp	6acc6eb1

5. **Fusiona la rama feature con la rama principal**. Para ello asegúrate de estar ubicado en la rama principal (main) y fusiona los cambios (merge) de la rama feature dentro de la principal.

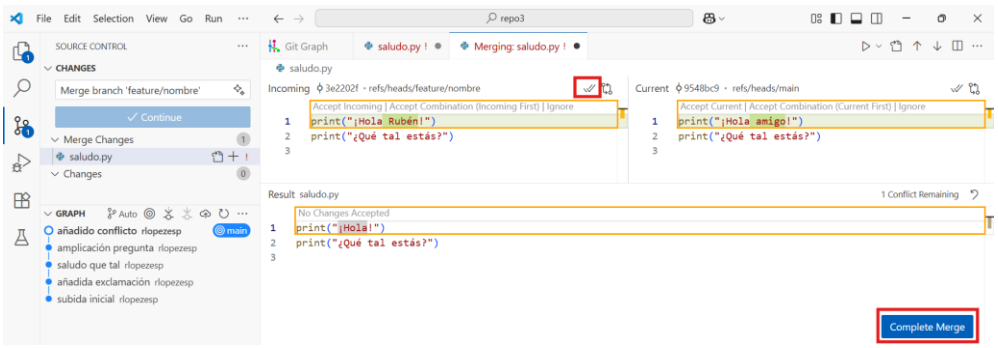




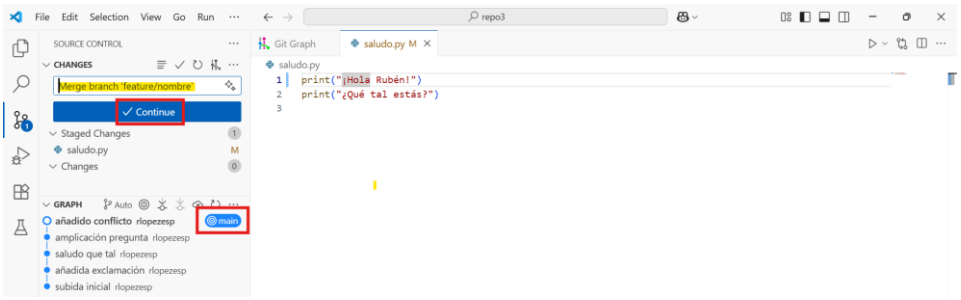
Como ves en la ventana emergente, el entorno de desarrollo nos avisa del conflicto que se acaba de producir y nos insta a solucionarlo antes de realizar el commit de la fusión.



6. Utilizaremos la herramienta de comparación del entorno y **resolveremos, de forma manual, el conflicto** que se ha generado. Para ello nos quedaremos con la porción de código que se corresponda con aquello que queremos desarrollar.



Por último, realizaremos un **commit** para integrar los cambios que hemos realizado manualmente.



Podemos ver el **historial de cambios** que nos ofrece la herramienta "Git graph" para comprobar visualmente el trabajo que acabamos de realizar.

Graph	Description	Date	Author	Commit
	main Merge branch 'feature/nombre'	18 Aug 2025 16:30	rlopezesp	fe6db7a7
	añadido conflicto	18 Aug 2025 16:22	rlopezesp	9548bc9d
	feature/nombre añadido nombre	18 Aug 2025 16:19	rlopezesp	3e2202f5
	amplicación pregunta	18 Aug 2025 00:03	rlopezesp	1247f8ed
	saludo que tal	17 Aug 2025 23:56	rlopezesp	b5aaa269
	añadida exclamación	17 Aug 2025 23:36	rlopezesp	362f81bc
	subida inicial	17 Aug 2025 23:35	rlopezesp	6acc6eb1

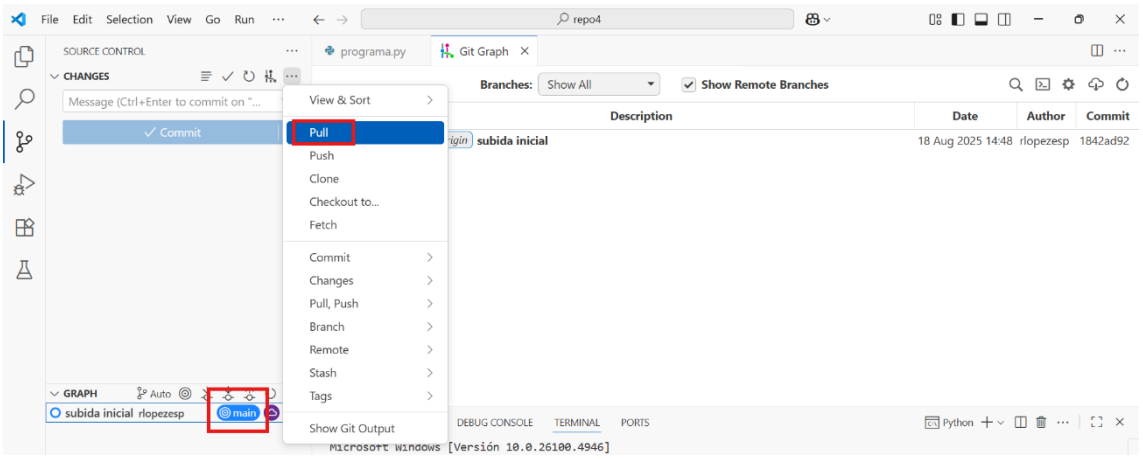
5.5. Ramas y repositorios remotos

En los apartados anteriores hemos trabajado con ramas, pero utilizando solamente un repositorio local, lo cual no es una situación demasiado común si estamos trabajando en equipo. Por ello, en este apartado vamos a trabajar con ramas, pero esta vez utilizando tanto el repositorio local como el remoto.

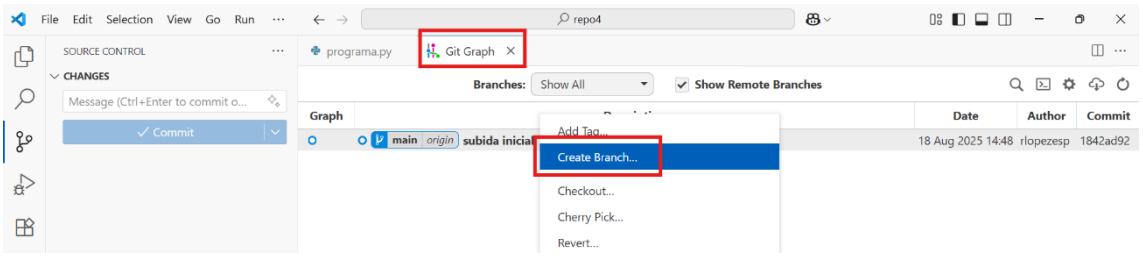
Además, usaremos el entorno de trabajo Visual Studio Code, simulando un flujo de trabajo posible dentro de un equipo de desarrollo.

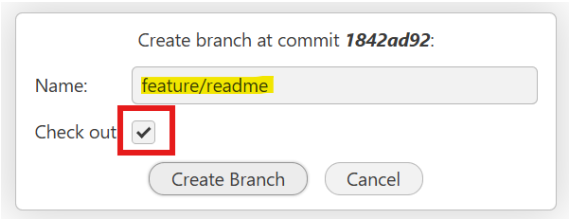
Un flujo de trabajo habitual trabajando con ramas y repositorios remotos podría ser el siguiente:

1. **Sincroniza tu rama principal** con el repositorio remoto: Para ello asegúrate de estar en la rama principal (main) y descarga los últimos cambios del repositorio remoto. Para ello puedes utilizar los comandos que vimos anteriormente o las herramientas que te ofrece tu entorno de desarrollo.

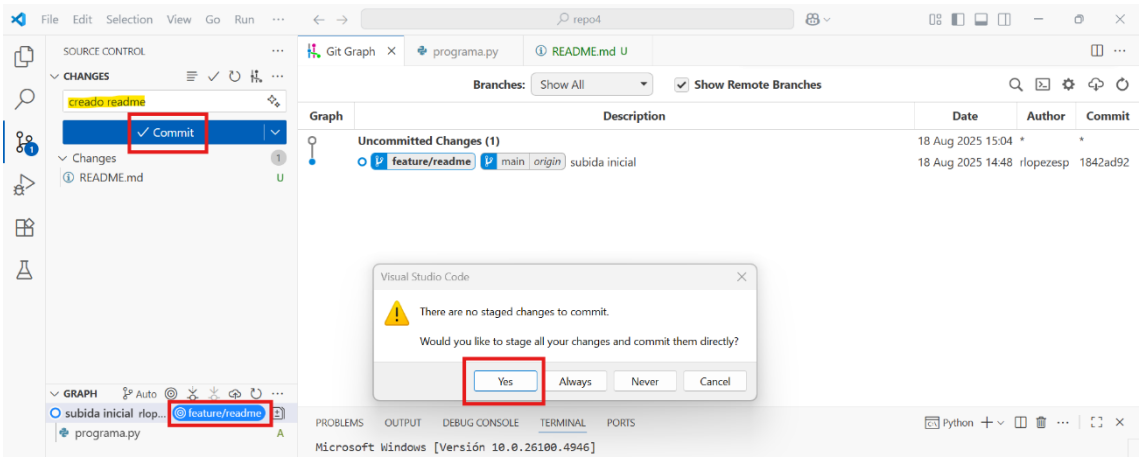


2. **Crea la nueva rama**: crea una nueva rama y asegúrate de estar ubicado en ella antes de continuar con el trabajo.

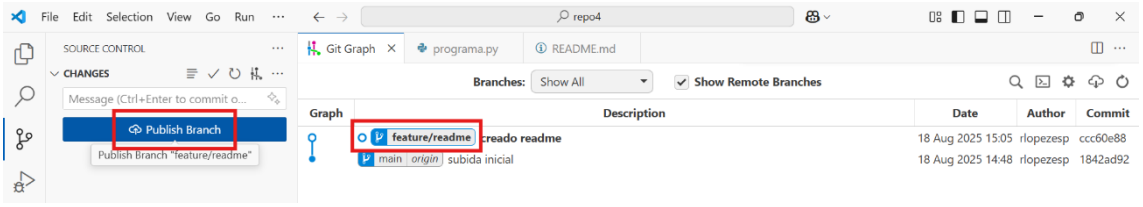




3. Desarrolla los **cambios y haz commit** en la nueva rama: modifica o crea los ficheros que necesites. Asegúrate de estar en la nueva rama y haz commit de los cambios (recuerda que antes debes pasar los cambios al staging area).

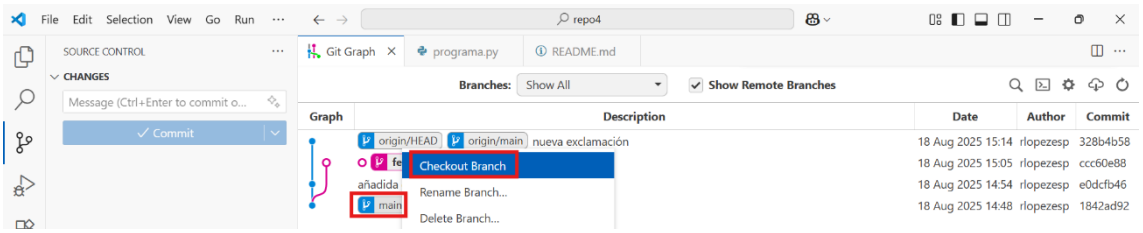


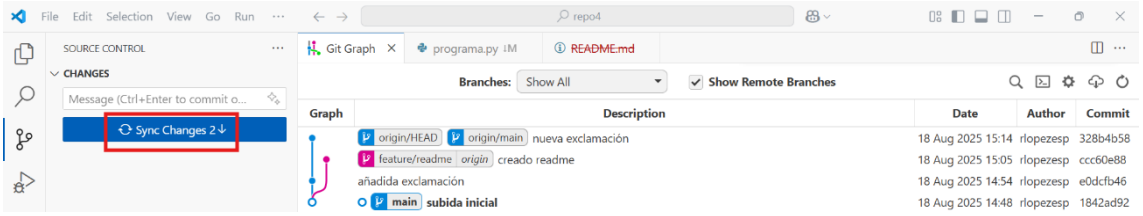
4. **Sube tu rama** al repositorio remoto.

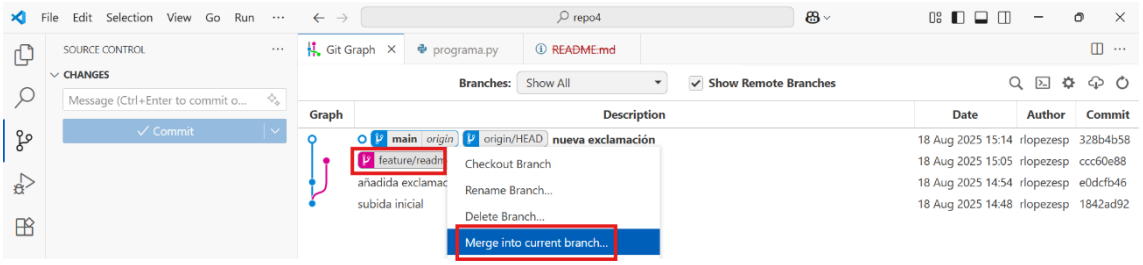


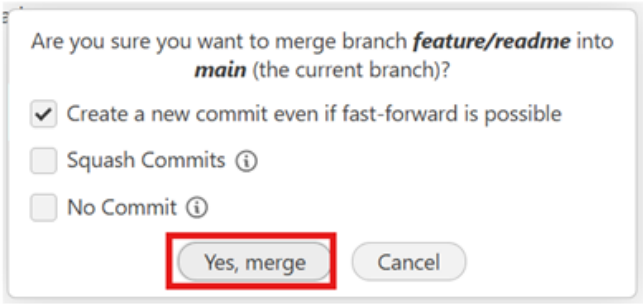
En este punto podríamos seguir realizando commits sobre la rama feature y enviarlos después a dicha rama del repositorio remoto. Esto será bastante habitual si varias personas trabajan sobre la misma feature.

5. Una vez terminado el desarrollo de la nueva característica toca **fusionar la rama feature con la rama principal**. Para ello tendrás que moverte a la rama main (ya que es la receptora de los cambios), asegurarte que está actualizada (**realizando un pull** o sincronizando los cambios) y realizar la fusión de la rama feature con la rama main.

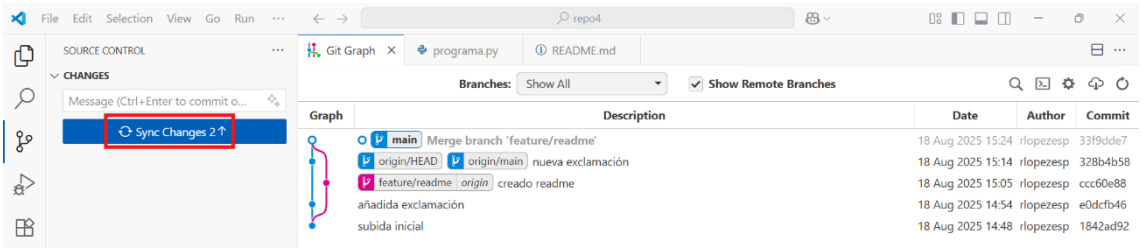








6. **Sube los cambios de la rama main** de tu repositorio remoto: Como puedes ver en el gráfico de la extensión “Git graph” al fusionar la rama feature con tu rama main has hecho los cambios en local, uniendo ambas ramas en un nuevo commit. Ahora solo falta subir dichos cambios a tu repositorio remoto. Para ello puedes realizar un Push o Sincronizar los cambios.



6. Más sobre Git y GitHub

6.1. Buenas prácticas de uso de Git

A continuación, una breve lista con algunos consejos y buenas prácticas a la hora de usar Git:

- Realizar commits atómicos, que representen una sola unidad lógica de cambio.
- Es mejor realizar pequeños commits con frecuencia que commit grandes pero cada mucho tiempo.
- Evitar commits con cambios no relacionados.
- Utilizar mensajes de commit descriptivos y concisos (máximo 50 caracteres).
- Escoger nombres de ramas descriptivos, añadiendo el prefijo feature, bug, experiment, etc.
- Mantener el historial de cambios limpio y organizado.
- Usar ramas para trabajar en nuevas características o correcciones de errores.
- Realizar pruebas antes de hacer commit para evitar introducir errores.
- No hagas commits de archivos autogenerados. Utilizar el fichero .gitignore para para evitar subir dichos archivos.
- Aunque esté permitido, no se debe usar Git como repositorio de imágenes o gestor documental, está más orientado a control de versiones sobre proyectos relacionados con el desarrollo de software, y tecnología.

6.2. Gitignore: Ignorar archivos y directorios

El archivo **.gitignore** (con un punto delante) es utilizado para especificar los archivos y directorios que Git debe ignorar al realizar commits. Dicho archivo se debe ubicar en la raíz del proyecto. Esto es tremendamente útil para excluir archivos temporales, archivos generados automáticamente y otros que no deben estar en control de versiones. Esto nos pueden

Ejemplo de contenido para el archivo .gitignore:

```
# Ignorar archivos temporales
*.tmp

# Ignorar carpeta "log" y todo su contenido
log/*

# Ignorar archivos sensibles
password.txt
credentials.txt
private_key.pem
```

Existe una página llamada gitignore.io que, indicándole algunas tecnologías o lenguajes de programación que vayas a usar en tu proyecto, te genera automáticamente el contenido del archivo .gitignore muy completo.

6.3. Etiquetas (tags)

Las etiquetas en Git son referencias a un commit específicos en la historia de un repositorio. Se utilizan comúnmente para **marcar versiones importantes** de un proyecto, como versiones estables o lanzamientos.

La convención de versionado semántico (o *SemVer* en inglés) es un estándar que define cómo asignar versiones a las liberaciones de software. La versión sigue el formato **X.Y.Z**, donde:

- **X** representa la versión mayor (major version): Se incrementa cuando se hacen cambios incompatibles en la API o en el comportamiento del software. Dicho de otro modo, cuando la nueva versión tiene poco que ver con la anterior y no son retrocompatibles.

- **Y** representa la versión menor (minor version): Se incrementa cuando se añaden nuevas características de una manera compatible con versiones anteriores.

- **Z** representa la versión de parche (patch version): Se incrementa para correcciones de errores que son compatibles con versiones anteriores.

Además de estas versiones principales, se pueden añadir identificadores para prelanzamientos y metadatos de construcción. Un número de versión podría verse así: **1.2.3-alpha+001**.

Cuando un software está en una versión que comienza por 0, como por ejemplo 0.0.1, generalmente indica que el software está en una fase temprana de desarrollo y no se considera estable para uso general. Se utilizan comúnmente para indicar versiones alfa o beta, donde el software aún está en desarrollo activo y no todas las características planeadas han sido implementadas.

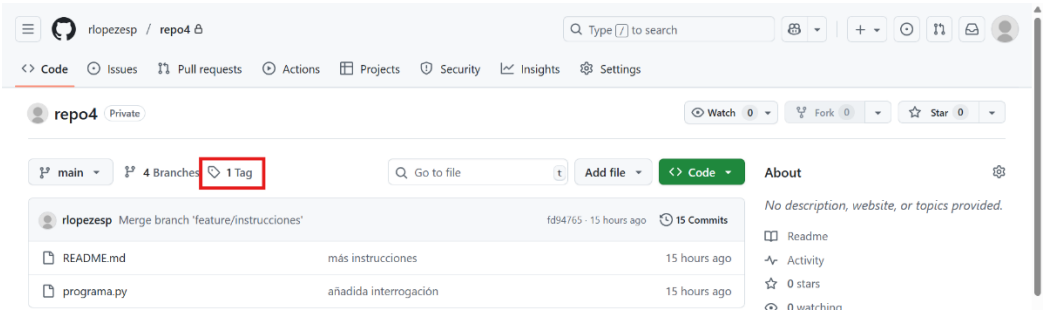
Ejemplo de **creación de etiqueta** en el repositorio local:

```
git tag -a v1.0.0 -m "Versión de lanzamiento 1.0.0"
```

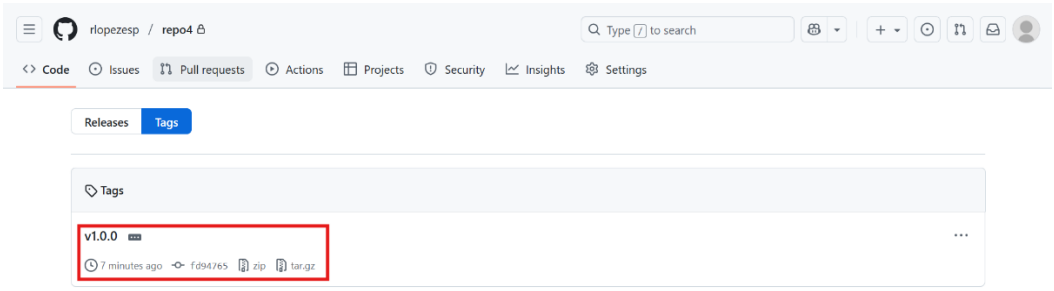
Ejemplo de **subida de la etiqueta** local al repositorio remoto:

```
git push origin v1.0.0
```

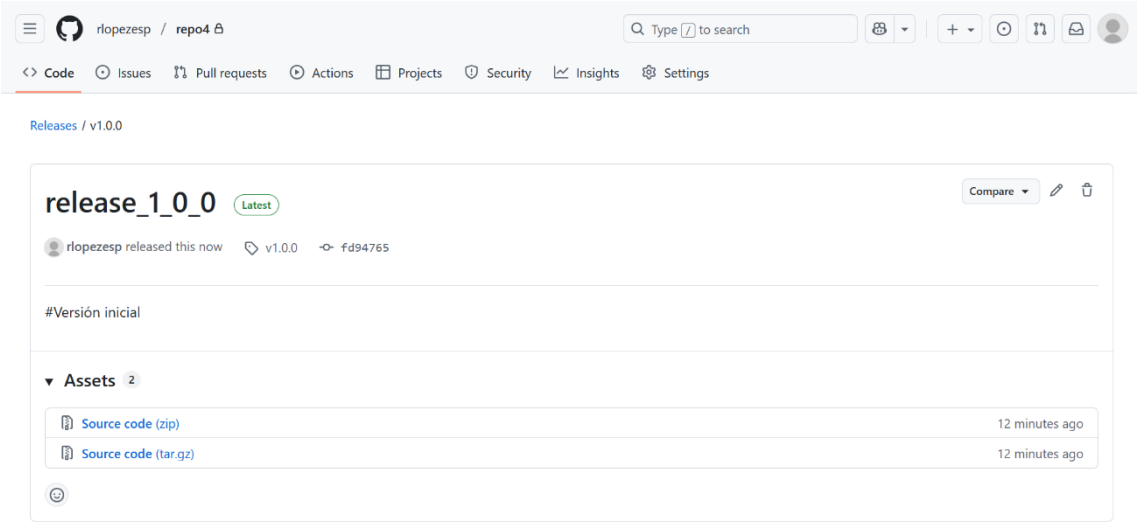
Una vez subida la etiqueta al repositorio remoto (GitHub), puedes consultar la misma abriendo tu repositorio en tu cuenta de GitHub y pulsando sobre la pesataña Tag:



Al entrar podrás ver todos los tags creados en tu repositorio remoto e, incluso, bajarte un fichero con el contenido del repositorio en el momento de la creación de dicho tag.



También podrías crear “**Releases**”. Un release de GitHub es un tag de Git enriquecido con notas de la versión y archivos binarios, diseñado para empaquetar y distribuir versiones de software de forma oficial.



Otros comandos útiles son:

- Ver los tags existentes: git tag
- Ver la información detallada de un tag anotado (-a): git show v1.0.0
- Eliminar un tag localmente: git tag -d v1.0.0
- Eliminar un tag en el repositorio remoto (primero eliminas el tag localmente y luego lo empujas al remoto): git push origin --delete v1.0.0

6.4. Estrategias de ramas en Git

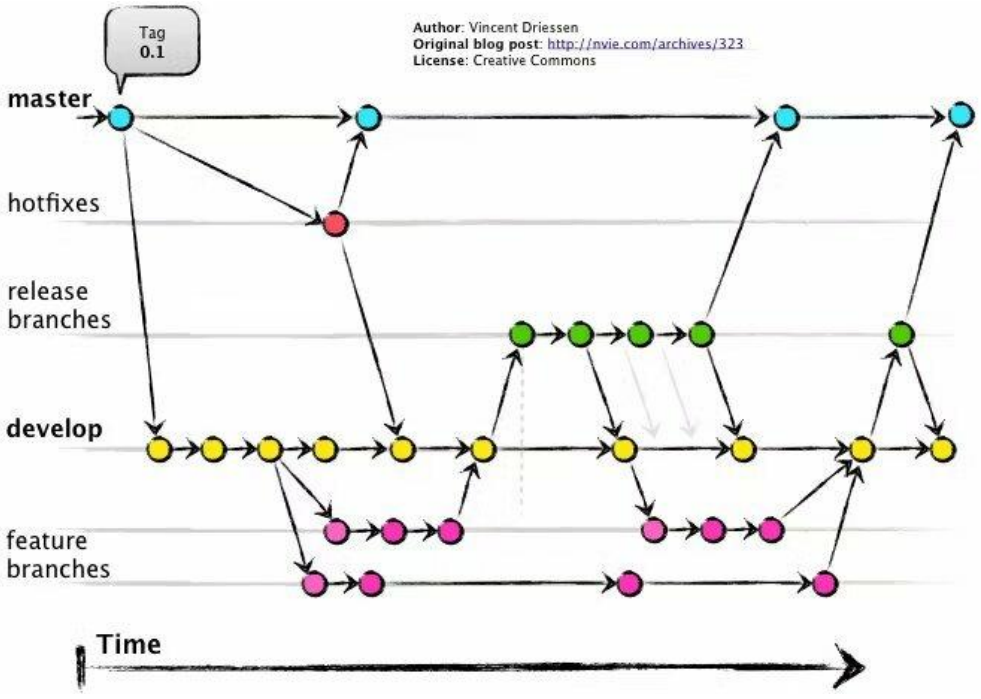
Las estrategias o flujos de trabajo de ramas en Git son guías que los equipos de desarrollo utilizan para organizar el código, colaborar de manera efectiva y gestionar las versiones de una aplicación. No hay una "mejor" estrategia única, la elección depende del tamaño del equipo, la complejidad del proyecto y el modelo de entrega de software (por ejemplo, despliegue continuo o lanzamientos programados).

Algunas de las estrategias más comunes son:

- **Git Flow**: creado por Vincent Driessen, es una de las estrategias más famosas y antiguas. Es ideal para proyectos con ciclos de lanzamiento definidos y que necesitan soporte para múltiples versiones. Ya que presenta un modelo más complejo y estructurado.

  Ramas principales: Tiene dos ramas principales de larga duración:
  - **main**: Representa el código que está en producción. Solo los commits de versiones estables llegan a esta rama.
  - **develop**: Es la rama de integración. Contiene el historial de desarrollo más reciente con las características ya fusionadas.

  Ramas de soporte: Además de las principales, utiliza ramas temporales con propósitos específicos:
  - feature/*: Para desarrollar nuevas características. Nacen de develop y se fusionan de nuevo en develop.
  - release/*: Para preparar una nueva versión. Nacen de develop y se fusionan en main y develop cuando están listas.
  - hotfix/*: Para correcciones urgentes en la producción. Nacen de main y se fusionan en main y develop.



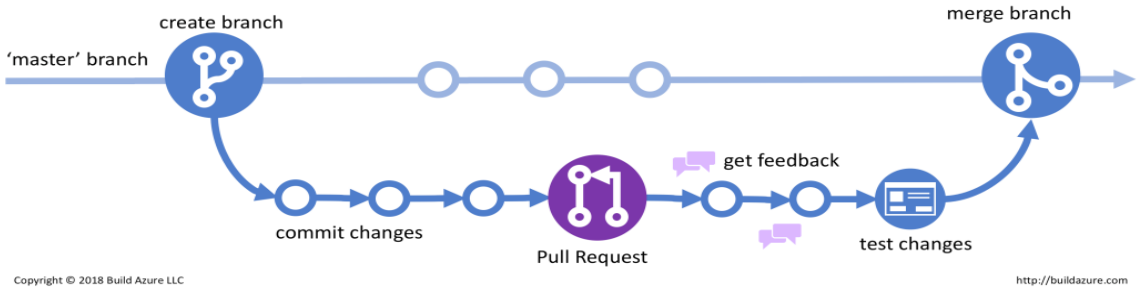
- **GitHub Flow**: Esta es una de las estrategias más simples y populares. Creada por la propia GitHub y pensada especialmente para equipos que practican la integración y entrega continuas (CI/CD). Se basa en la creación de Pull Request (PR) que abrirán una revisión del nuevo código antes de integrarse con la rama principal.

  Ramas principales: Solo tiene una rama principal de larga duración, que será desplegada (producción):

  - **main**: Representa el código que está en producción.

  El flujo de trabajo utilizando esta estrategia sería el siguiente:

  1. Cualquier nueva característica o corrección se crea en una nueva rama a partir de main (rama feature).
  2. Se trabaja en esa rama y se realizan los commits correspondientes.
  3. Cuando el trabajo está listo, se abre un Pull Request (PR) para iniciar una revisión de código.
  4. Una vez aprobado y fusionado el PR, los cambios se incorporan a main y se pueden desplegar inmediatamente.



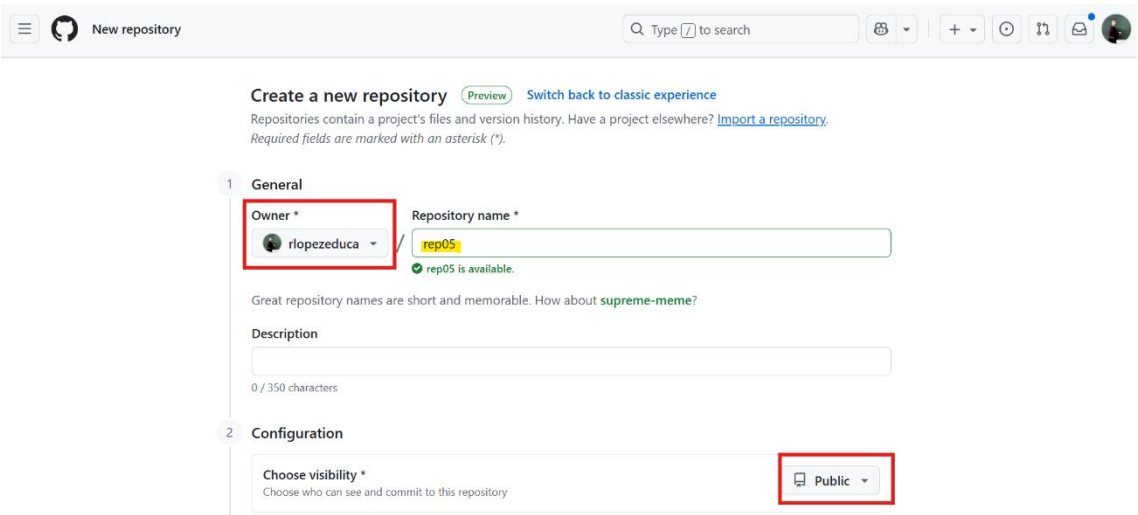
6.5. Colaboración en equipo (fork y pull requests)

Siguiendo la **estrategia GitHub Flow** de la que hemos hablado en apartado anterior vamos a trabajar en un proyecto con otros colaboradores. Para este tipo de proyectos existen dos posibilidades:

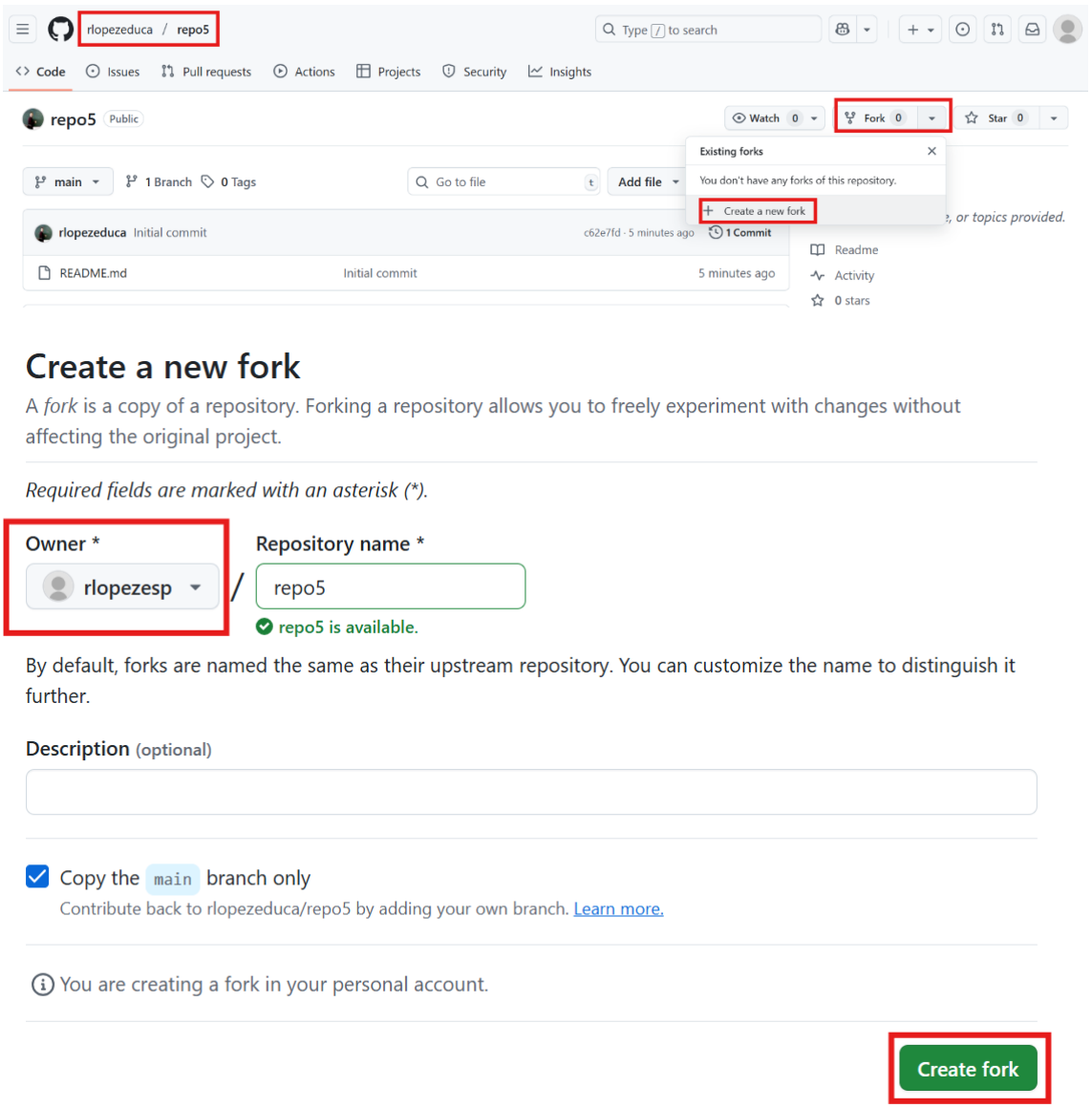
- Un repositorio privado, para un desarrollo de alguna aplicación, en el que varios desarrolladores tienen permiso para trabajar sobre ciertas ramas, pero no sobre la rama principal. Por lo tanto, para realizar cambios sobre dicha rama tendrían que realizar los cambios en una **rama feature** y, después, solicitar un **pull request** para fusionar su rama sobre la rama principal.

- Un repositorio público, para un desarrollo open source por ejemplo, en el que los desarrolladores externos no tienen permisos sobre ninguna rama del repositorio. En este caso es común utilizar **forks** y **pull requests** para contribuir al proyecto principal de manera organizada. Un **fork** es una copia de un repositorio de otra persona en tu cuenta.

Vamos a ver un ejemplo del segundo caso. En primer lugar, necesitamos un repositorio público. Dicho repositorio habrá sido creado con una cuenta de GitHub diferente a la que después utilizaremos para realizar el fork y el pull request.



Ahora, desde otra cuenta de GitHub, entramos a la página web del repositorio original y **realizamos un fork** del mismo. Veremos lo siguiente:

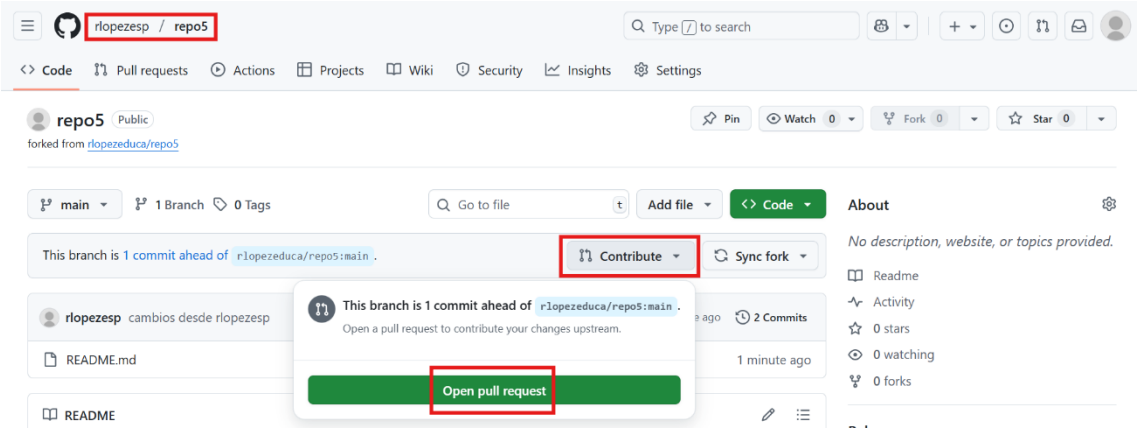


A partir de este momento dispondremos de un nuevo repositorio en nuestra cuenta de GitHub que estará "**enlazado**" con el repositorio original a que acabamos de hacer un fork.

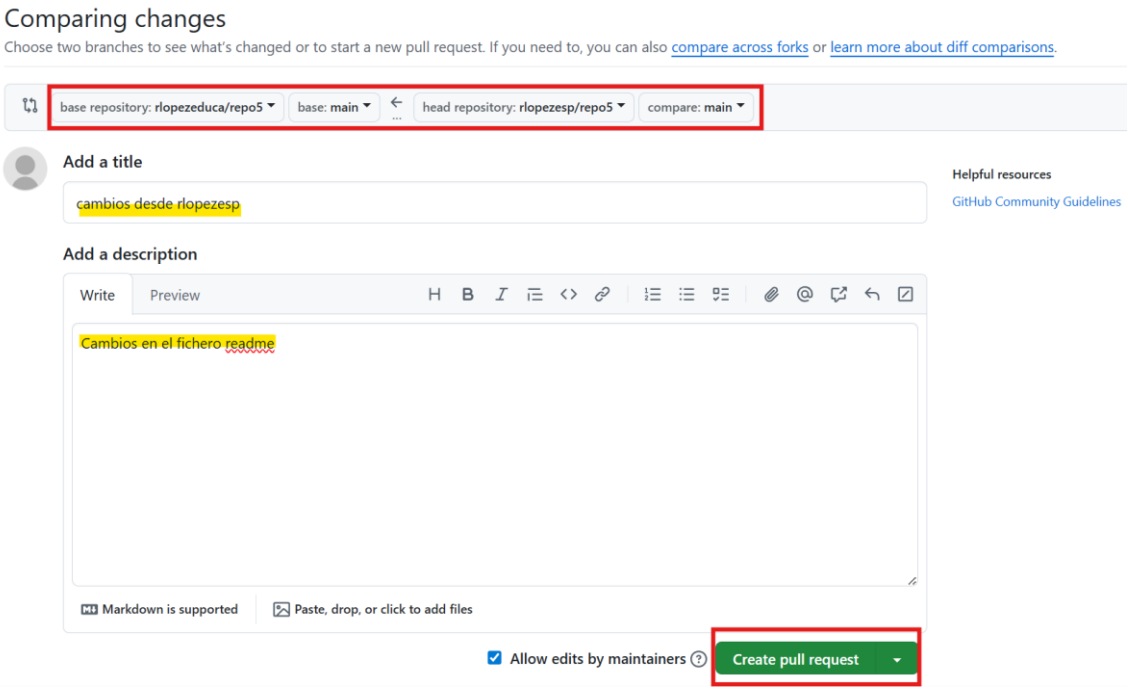
Los próximos pasos serán los siguientes:

- **Clonar** este último repositorio en nuestro equipo.
- Realizar los cambios que se deseen y hacer **commit** sobre la rama principal de tu repositorio.

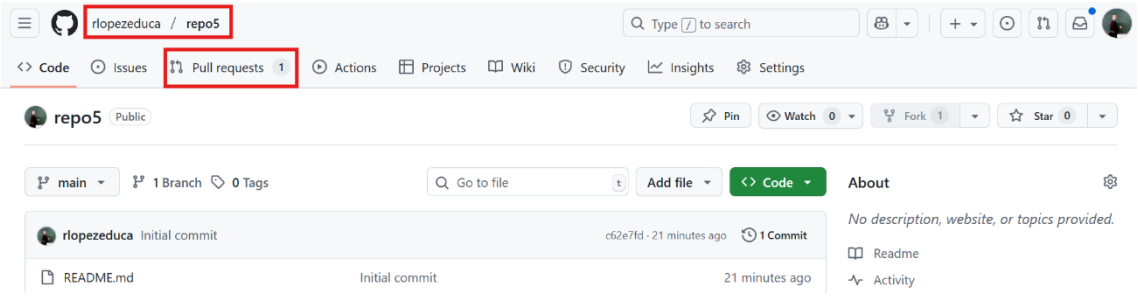
- Subir los cambios a tu repositorio remoto mediante un **push**.
- **Realizar un pull request** de tu repositorio sobre el repositorio original. Para ello entra en tu cuenta de GitHub y abre tu repositorio. Verás una página similar a la siguiente:

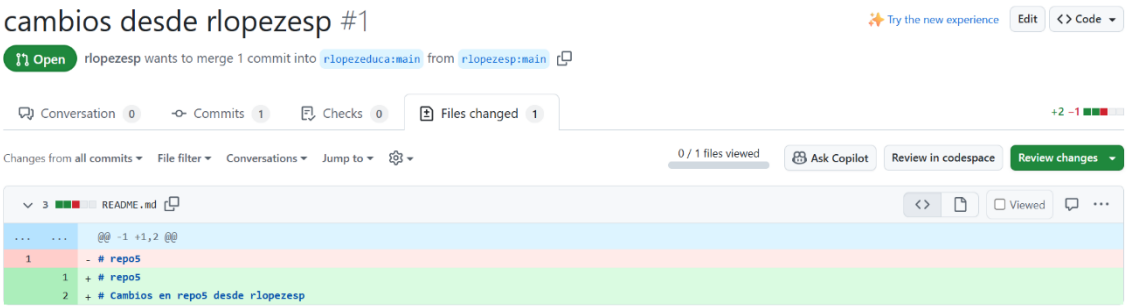


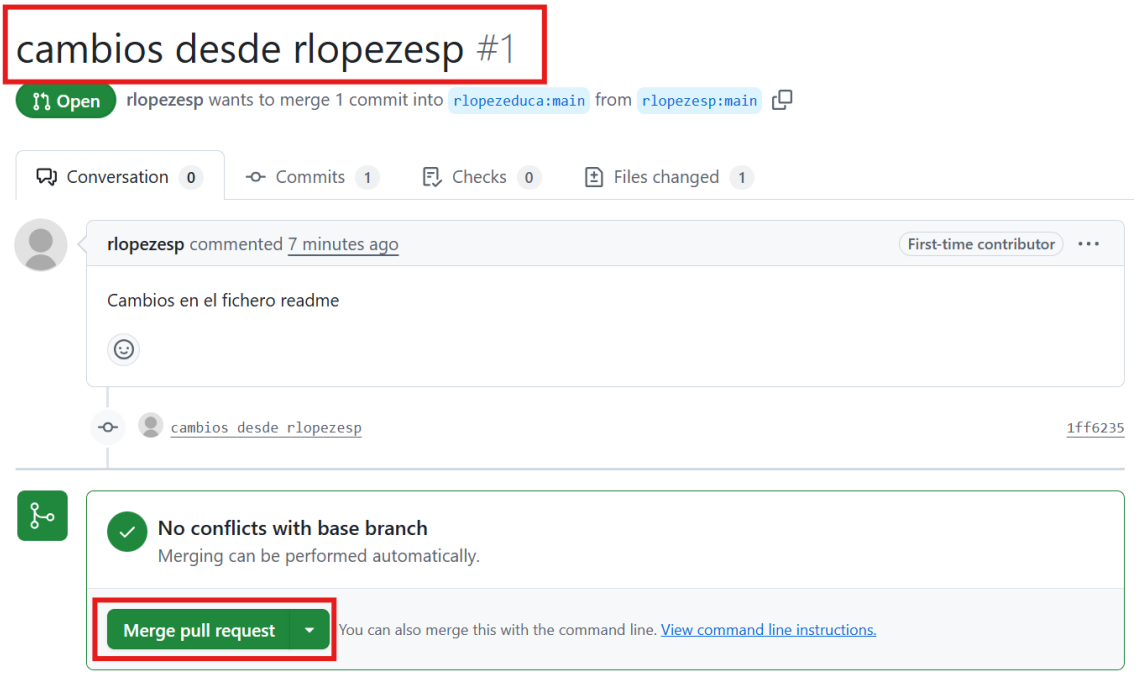
En la siguiente ventana comprueba que las ramas origen y destino son las correctas y rellena la descripción de la solicitud explicando el cambio que estás proponiendo sobre el repositorio original:



- Para terminar, solo faltaría entrar en el repositorio original y **aprobar el pull request** que hemos recibido:



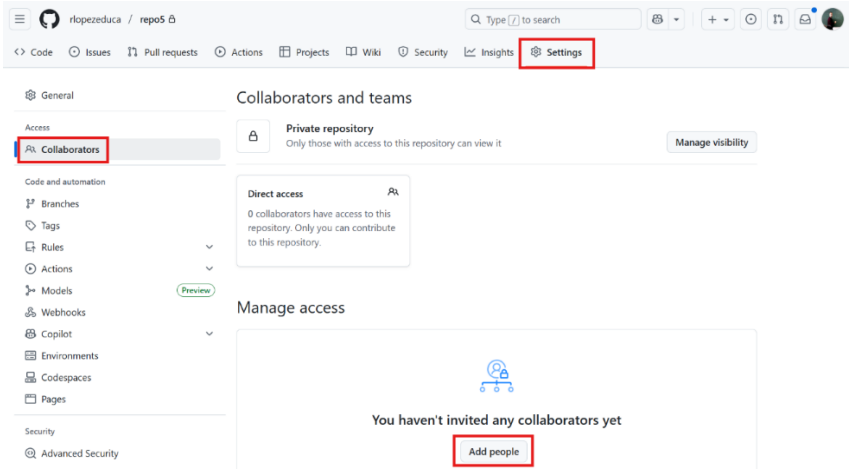




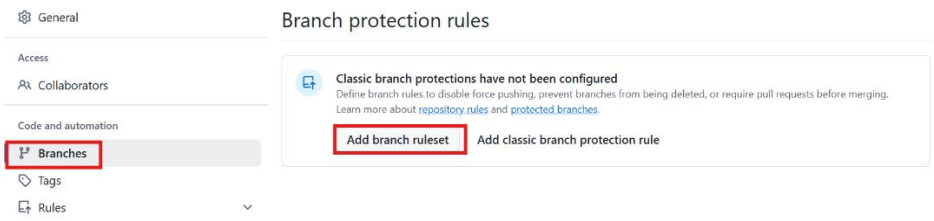
Esta manera de trabajar en un proyecto colaborativo que acabamos de ver ofrece mayor control sobre los cambios, ya que todos los cambios los vas a realizar sobre tu fork o copia del proyecto de otras personas. Por lo tanto no existe riesgo de romper nada. Solo tienes que hacer los cambios que consideres oportunos y, cuando hayas finalizado, puedes hacer un pull request para fusionar dichos cambios con el repositorio original.

Otra posibilidad, como se comentó anteriormente, sería disponer de un **repositorio privado** y **añadir colaboradores**. Estos colaboradores podrán realizar cambios sobre cierta rama, por ejemplo la rama develop, o crear nuevas ramas. Pero no podran hacer cambios sobre la rama principal (main). Para incorporar sus cambios al a rama principal tendrían que realizar un pull request de manera similar a lo explicado en el caso anterior con el fork.

Para añadir colaboradores solo tendremos que abrir nuestro repositorio, entrar en “Settings”, “Collaborators” y pulsar sobre el botón “Add people”.



También sería importante que nuestro repositorio tuviera implementadas **Reglas de protección de ramas**. De esta manera podríamos permitir a los colaboradores hacer cambios en la rama develop pero no en la rama main. Para esta última habría que solicitar los cambios mediante una pull request.



6.6. GitHub Pages

GitHub Pages es un servicio de alojamiento de **sitios web estáticos** proporcionado por GitHub. En esencia, permite a los usuarios publicar páginas web directamente desde un repositorio de GitHub. Es la forma más sencilla de crear un sitio web personal, de un proyecto o de una organización, sin necesidad de tener un servidor o una base de datos.

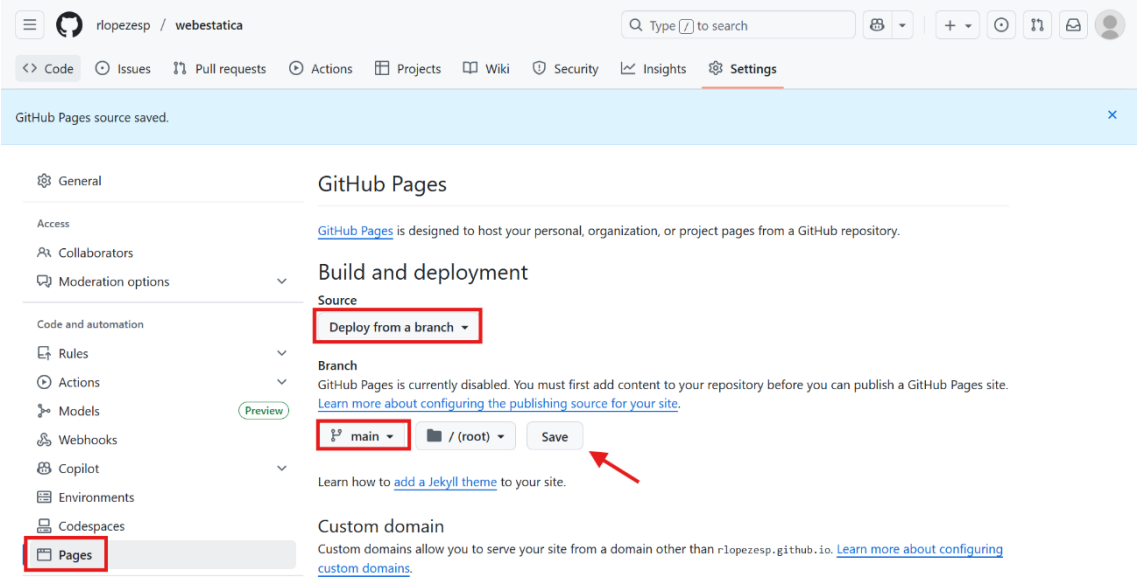
Es una herramienta ideal para alojar páginas web estáticas, es decir, páginas web que solo necesitan HTML, CSS y JavaScript, como documentación de proyectos, blogs personales o portafolios en línea.

Crear una página web estática con GitHub Pages es un proceso muy sencillo:

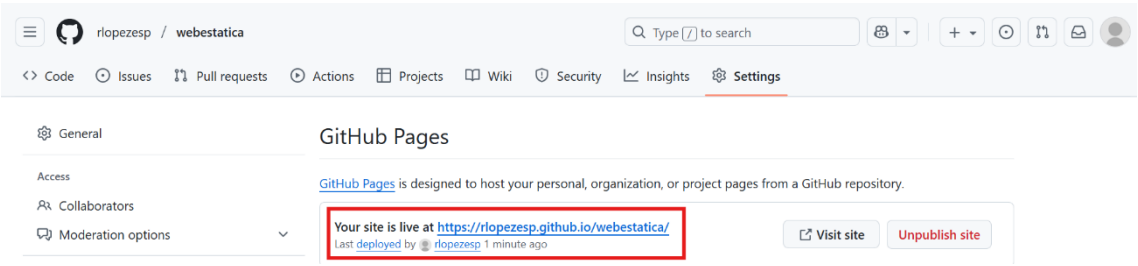
1. Crea un nuevo repositorio en GitHub. Debe tener **visibilidad pública**.
2. Clona dicho repositorio en tu equipo.
3. Crea un archivo llamado **index.html**, implementa tu página web y sube los cambios al repositorio remoto. En este paso puedes apoyarte en GitHub Copilot:



4. Habilita GitHub Pages en tu repositorio remoto. Para ello:
   - Abre la pestaña **Settings** de tu repositorio.
   - En el menú de la izquierda, selecciona **Pages**.
   - En "Source" o "Build and deployment", elige la **rama (main)** y la carpeta (/root o /docs) que contiene los archivos de tu sitio web.
   - Haz clic en **Save**.



5. Accede a tu sitio. Si esperas unos segundos GitHub publicará tu sitio y te mostrará la URL del mismo.



6.7. Learn Git Branching

Por último, os propongo la herramienta “Learn Git Branching”. Esta web es una aplicación interactiva y un tutorial diseñado para ayudar a los principiantes a aprender y practicar los conceptos principales de Git de una manera visual y práctica.

Puedes acceder a la misma en el siguiente enlace:

https://learngitbranching.js.org/?locale=es_ES